HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

_____ o0o _____



BÀI TẬP LỚN

Đề tài: Hệ thống IoT giám sát sức khỏe người bệnh

Môn học: IoT và Ứng dụng
Số thứ tự nhóm: 06

Lưu Trung Kiên MSSV: B21DCCN071
Cao Bá Hiếu MSSV: B21DCCN347
Mạc Quang Đạt MSSV: B21DCCN215
Vũ Lê Hùng MSSV: B21DCCN059

Giảng viên hướng dẫn: Kim Ngọc Bách

HÀ NỘI, 10/2024

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN VỀ BÀI TẬP LỚN 1

1.1 Giới thiệu đề tài 1

1.2 Lý do chọn đề tài 1

1.3 Mục tiêu đề tài 2

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 3

2.1 Tổng quan về IoT 3

2.1.1 Khái niệm IoT 3

2.1.2 Lịch sử hình thành 3

2.1.3 Kiến trúc một hệ thống IoT 4

2.2 Hệ thống IoT trong giám sát sức khỏe người bệnh 4

2.2.1 Tổng quan 4

2.2.2 Nguyên lý hoạt động 5

2.3 Giới thiệu về phần mềm Adruino 5

2.3.1 Khái niệm 5

2.3.2 Ứng dụng 6

2.3.3 Cấu trúc của một chương trình Arduino 6

2.4 Giới thiệu về Arduino Uno R3 7

2.4.1 Tổng quan 7

2.4.2 Thông số kỹ thuật 7

2.4.3 Quy trình hoạt động cơ bản 8

2.5 Giới thiệu về ESP8266 NodeMCU 9

2.5.1 Tổng quan 9

2.5.2 Thông số kỹ thuật 9

2.5.3 Quy trình hoạt động 10

2.6 Giới thiệu về cảm biến LM35 10
2.6.1 Tổng quan 10
2.6.2 Thông số kỹ thuật 11
2.6.3 Quy trình hoạt động 11
2.7 Giới thiệu về DHT11 12
2.7.1 Tổng quan 12
2.7.2 Thông số kỹ thuật 12
2.7.3 Quy trình hoạt động 13
2.8 Giới thiệu về Pulse Sensor 13
2.8.1 Tổng quan 13
2.8.2 Thông số kỹ thuật 14
2.8.3 Quy trình hoạt động 14
CHƯƠNG 3. PHÂN TÍCH YÊU CẦU 16
3.1 Mục đích 16
3.2 Phạm vi hệ thống 17
3.3 Yêu cầu hệ thống 17
3.4 Yêu cầu chi tiết theo nhóm người dùng 18
3.4.1 Quản trị viên nhân viên y tế 18
3.4.2 Bệnh nhân 18
3.4.3 Thiết bị 18
3.5 Sơ đồ Use Case tổng quát 19
CHƯƠNG 4. THIẾT KẾ HỆ THỐNG 21
4.1 Mục tiêu hệ thống 21
4.2 Hành vi và các hoạt động của hệ thống 21
4.3 Đặc tả tiến trình 23
4.4 Đặc tả mô hình miền 24

4.5 Đặc tả mô hình thông tin 26
4.6 Thông số dịch vụ 27
4.7 Đặc tả cấp độ IoT 28
CHƯƠNG 5. PHÂN TÍCH HỆ THỐNG 30
5.1 Kết nối thiết bị 30
5.2 Danh sách công việc cần thực hiện 30
5.2.1 Đọc và xử lý thông tin từ các sensor 30
5.2.2 Giao tiếp với MQTT server 31
5.2.3 Thiết kế và cài đặt giao diện 32
5.2.4 Cài đặt, kết nối và các phương thức xử lý liên quan tới database 37
5.2.5 Cài đặt MQTT Client 39
5.2.6 Các API cho các chức năng của nhân viên quản trị y tế 39
5.2.7 Các API cho các chức năng của bệnh nhân 41
5.3 Bảng phân chia công việc 45
5.3.1 Phân công ban đầu 45
5.3.2 Theo dõi tiến trình 45
CHƯƠNG 6. ĐÁNH GIÁ KẾT QUẢ DỰ ÁN, KẾT LUẬN 47
6.1 Đánh giá kết quả dự án 47
6.1.1 Các yêu cầu chi tiết theo nhóm người dùng đã hoàn thành 47
6.1.2 Các yêu cầu chi tiết theo nhóm người dùng chưa hoàn thành 48
6.1.3 Kết luận 49
6.1.4 Rút ra bài học kinh nghiệm 49
6.1.5 Định hướng phát triển tương lai 50
TÀI LIỆU THAM KHẢO 52

DANH MỤC HÌNH VẼ

Hình 2.1 Adruino Uno R3 7
Hình 2.2 Esp 8266 nodeMCU 9
Hình 2.3 LM35 11
Hình 2.4 DHT11 12
Hình 2.5 Pulse Sensor 14

Hình 3.1 Sơ đồ Use Case Tổng Quát 19

Hình 4.1 Đặc tả tiến trình 23
Hình 4.2 Đặc tả mô hình miền 24
Hình 4.3 Đặc tả mô hình thông tin 26
Hình 4.4 Thông số dịch vụ 27
Hình 4.5 Đặc tả cấp độ IoT 28

Hình 5.1 Sơ đồ kết nối thiết bị 30
Hình 5.2 Giao diện đăng nhập cho nhân viên quản trị y tế 32
Hình 5.3 Giao diện trang chính của nhân viên quản trị y tế 33
Hình 5.4 Giao diện danh sách các bệnh nhân 33
Hình 5.5 Giao diện lịch sử đo của một bệnh nhân cụ thể 33
Hình 5.6 Giao diện tổng quan ban đầu 34
Hình 5.7 Giao diện đăng nhập/đăng ký 34
Hình 5.8 Giao diện trang chính bệnh nhân 34
Hình 5.9 Giao diện trang thay đổi thông tin bệnh nhân 35
Hình 5.10 Giao diện trang chỉ số sức khỏe bệnh nhân 35
Hình 5.11 Giao diện trang hiện lịch sử đo chỉ số cơ thể bệnh nhân 35
Hình 5.12 Giao diện đồ thị đo chỉ số bệnh nhân 36
Hình 5.13 Giao diện trang hiện lịch sử đo chỉ số phòng của bệnh nhân . . 36
Hình 5.14 Giao diện đồ thị đo chỉ số phòng bệnh 36
Hình 5.15 Sơ đồ cơ sở dữ liệu 37

CHƯƠNG 1. TỔNG QUAN VỀ BÀI TẬP LỚN

1.1 Giới thiệu đề tài

Trong thời đại công nghệ số, sức khỏe con người ngày càng được quan tâm. Việc theo dõi các chỉ số sinh tồn như nhiệt độ cơ thể và nhịp tim thường xuyên là một trong những cách hiệu quả để phát hiện sớm các dấu hiệu bất thường và ngăn ngừa bệnh tật. Tuy nhiên, các phương pháp truyền thống như sử dụng nhiệt kế thủy ngân hoặc đếm nhịp tim thủ công thường không mang lại độ chính xác cao và gây bất tiện cho người dùng.

Trong bối cảnh công nghệ thông tin tại Việt Nam đang phát triển mạnh mẽ, các giải pháp IoT ngày càng được ứng dụng rộng rãi vào nhiều lĩnh vực khác nhau. Trong đó, lĩnh vực y tế cũng không ngoại lệ. Với tinh thần sáng tạo và đổi mới, nhóm lựa chọn nghiên cứu và phát triển hệ thống IoT giám sát sức khỏe, nhằm mang đến những giải pháp chăm sóc sức khỏe hiệu quả và tiện lợi hơn. Hệ thống này hoạt động dựa trên việc thu thập dữ liệu từ các cảm biến giúp theo dõi liên tục các chỉ số sinh tồn quan trọng như nhiệt độ và nhịp tim từ xa giúp các bác sĩ giám sát sức khỏe người bệnh tốt hơn.

1.2 Lý do chọn đề tài

Trong khuôn khổ của một đề tài nghiên cứu và với sự hướng dẫn tận tình của thầy Kim Ngọc Bách, nhóm chúng em đã quyết định tập trung vào việc phát triển "Hệ thống IoT giám sát sức khỏe người bệnh".

Sự lựa chọn này được đưa ra dựa trên những lý do sau:

- **Nhu cầu thực tế ngày càng cao:** Với sự gia tăng của dân số già và các bệnh mãn tính, nhu cầu về các giải pháp chăm sóc sức khỏe tại nhà hiệu quả và tiện lợi trở nên cấp thiết hơn bao giờ hết. Hệ thống IoT chúng em xây dựng hứa hẹn sẽ đáp ứng được nhu cầu này, giúp người bệnh tự theo dõi sức khỏe tại nhà và giảm bớt gánh nặng cho hệ thống y tế.
- **Tiềm năng ứng dụng rộng rãi:** Công nghệ IoT đang ngày càng phát triển và được ứng dụng vào nhiều lĩnh vực khác nhau. Việc áp dụng IoT vào lĩnh vực y tế, đặc biệt là trong giám sát sức khỏe người bệnh, mở ra nhiều triển vọng ứng dụng rộng rãi, từ việc chăm sóc người cao tuổi đến việc theo dõi sức khỏe của bệnh nhân mãn tính.
- **Khả năng kết hợp kiến thức đa ngành:** Đề tài này đòi hỏi chúng em phải kết hợp kiến thức từ nhiều lĩnh vực khác nhau như điện tử, lập trình và truyền thông dữ liệu. Điều này giúp chúng em rèn luyện kỹ năng làm việc nhóm, khả

năng tổng hợp thông tin và giải quyết vấn đề một cách toàn diện.

- **Đóng góp vào sự phát triển của cộng đồng:** Chúng em tin rằng hệ thống IoT này sẽ góp phần nâng cao chất lượng cuộc sống của người bệnh, giúp họ chủ động hơn trong việc quản lý sức khỏe của mình. Đồng thời, đây cũng là một đóng góp nhỏ bé của chúng em vào sự phát triển của ngành y tế Việt Nam.

1.3 Mục tiêu đề tài

Với những chức năng cơ bản như đo nhịp tim, đo nhiệt độ cơ thể người từ các cảm biến và gửi dữ liệu về ứng dụng để giám sát, chúng em mong muốn xây dựng một hệ thống đơn giản, dễ sử dụng nhưng vẫn đảm bảo độ chính xác cao.

Trong quá trình thực hiện, chúng em sẽ tập trung vào các mục tiêu sau:

- **Thiết kế và lựa chọn thiết bị:** Nghiên cứu và lựa chọn các loại cảm biến phù hợp, thiết kế mạch điện tử và xây dựng phần cứng của hệ thống.
- **Phát triển phần mềm:** Lập trình ứng dụng web để hiển thị dữ liệu, phân tích và theo dõi.
- **Đánh giá và cải tiến:** Thực hiện các thử nghiệm để đánh giá hiệu quả của hệ thống và liên tục cải tiến để nâng cao chất lượng.

Chúng em tin rằng với sự nỗ lực không ngừng, chúng em sẽ hoàn thành tốt đề tài này và đóng góp một phần nhỏ vào sự phát triển của công nghệ IoT tại Việt Nam.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về IoT

2.1.1 Khái niệm IoT

IoT (Internet of Things) hay Internet vạn vật là một mạng lưới bao gồm các thiết bị vật lý, phương tiện, thiết bị gia dụng và các đối tượng khác được kết nối với Internet. Những thiết bị này được trang bị các cảm biến, phần mềm, và khả năng kết nối mạng, cho phép chúng thu thập và trao đổi dữ liệu.

IoT hoạt động dựa trên việc kết nối các thiết bị vật lý với nhau thông qua mạng Internet. Các cảm biến trên thiết bị sẽ thu thập dữ liệu về môi trường xung quanh, sau đó truyền dữ liệu này lên một nền tảng trung tâm. Tại đây, dữ liệu sẽ được xử lý, phân tích và đưa ra các quyết định. Cuối cùng, các lệnh điều khiển sẽ được gửi trở lại các thiết bị để thực hiện các hành động cụ thể. Ví dụ, khi nhiệt độ trong phòng vượt quá ngưỡng cài đặt, cảm biến nhiệt độ sẽ gửi tín hiệu đến hệ thống điều hòa, và hệ thống sẽ tự động bật để làm mát căn phòng.

2.1.2 Lịch sử hình thành

Ý tưởng về việc kết nối các vật thể vật lý với nhau để tạo thành một mạng lưới thông tin đã xuất hiện từ những năm 1980. Tuy nhiên, phải đến những năm 1990, khi công nghệ vi mạch và truyền thông phát triển mạnh mẽ, ý tưởng này mới bắt đầu được hiện thực hóa.

- **Những năm 1980:** Các nhà nghiên cứu bắt đầu khám phá khả năng kết nối các thiết bị điện tử với nhau thông qua mạng. Tuy nhiên, do hạn chế về công nghệ, các ứng dụng thực tế còn rất hạn chế.
- **Những năm 1990:** Sự ra đời của World Wide Web đã tạo ra một cú hích lớn cho sự phát triển của IoT. Các nhà khoa học và kỹ sư bắt đầu nghiên cứu cách kết nối các đối tượng vật lý vào mạng Internet.
- **Những năm 1999:** Thuật ngữ "Internet of Things" chính thức được sử dụng bởi Kevin Ashton, khi ông làm việc tại Procter & Gamble. Ông đã hình dung ra một hệ thống sử dụng các thẻ RFID để theo dõi hàng hóa trong chuỗi cung ứng.
- **Những năm 2000:** Sự phát triển của các công nghệ như Wi-Fi, Bluetooth, và các cảm biến giá rẻ đã thúc đẩy sự phổ biến của IoT.
- **Hiện tại:** IoT đã trở thành một công nghệ cốt lõi, được ứng dụng rộng rãi trong nhiều lĩnh vực, từ nhà thông minh, sản xuất, nông nghiệp đến y tế.

2.1.3 Kiến trúc một hệ thống IoT

Kiến trúc IoT được đại diện cơ bản bởi 4 phần:

- Things (Thiết bị): Đây là lớp cơ bản nhất của hệ thống IoT, bao gồm các thiết bị vật lý được trang bị cảm biến và bộ truyền thông. Các thiết bị này có thể là: Cảm biến dùng để thu thập dữ liệu từ môi trường xung quanh như nhiệt độ, độ ẩm, ánh sáng, âm thanh, chuyển động, v.v. Bộ truyền động để thực hiện các hành động dựa trên dữ liệu nhận được, ví dụ như điều khiển đèn, mở cửa, điều chỉnh nhiệt độ hay các thiết bị đầu cuối là thiết bị mà người dùng tương tác trực tiếp, như điện thoại thông minh, máy tính bảng.
- Gateway (Trạm kết nối): Gateway đóng vai trò như một cầu nối giữa các thiết bị và mạng Internet. Các chức năng chính của gateway bao gồm: Nhận dữ liệu từ các thiết bị trong phạm vi kết nối, thực hiện các phép tính đơn giản, lọc dữ liệu và loại bỏ nhiễu, chuyển dữ liệu đến nền tảng đám mây hoặc trung tâm dữ liệu, cấu hình, cập nhật firmware và quản lý các thiết bị kết nối.
- Network and Cloud (Hạ tầng mạng): Đây là lớp nền tảng cho việc truyền tải và xử lý dữ liệu. Mạng bao gồm các mạng không dây (Wi-Fi, Bluetooth, LoRa, NB-IoT) và mạng có dây (Ethernet) để kết nối các thiết bị và gateway. Đám mây cung cấp không gian lưu trữ, khả năng tính toán và các dịch vụ phân tích dữ liệu lớn để xử lý lượng dữ liệu khổng lồ từ IoT.
- Services-creation and Solution Layers (Bộ phân tích và xử lý dữ liệu): Lớp này chịu trách nhiệm phân tích dữ liệu, xây dựng các ứng dụng và cung cấp các dịch vụ dựa trên dữ liệu thu thập được. Phân tích dữ liệu có thể bằng cách áp dụng các thuật toán học máy, trí tuệ nhân tạo để tìm ra các mẫu, xu hướng và mối quan hệ trong dữ liệu. Hay phát triển các ứng dụng trên nền tảng web hoặc di động để hiển thị dữ liệu, điều khiển thiết bị và cung cấp các dịch vụ giá trị gia tăng. Cũng có thể cung cấp các dịch vụ như dự báo, tối ưu hóa, tự động hóa dựa trên kết quả phân tích dữ liệu.

2.2 Hệ thống IoT trong giám sát sức khỏe người bệnh

2.2.1 Tổng quan

Trong thời đại công nghệ số phát triển mạnh mẽ, Internet of Things (IoT) đã tạo ra một cuộc cách mạng trong nhiều lĩnh vực, và y tế không phải là ngoại lệ. Hệ thống IoT giám sát sức khỏe người bệnh, với khả năng kết nối và truyền tải dữ liệu y tế một cách thông minh, đang dần trở thành một giải pháp đột phá trong việc chăm sóc sức khỏe từ xa.

Bằng việc kết nối các thiết bị y tế thông minh, cảm biến sinh trắc học và nền

tảng dữ liệu, IoT đã mở ra cánh cửa cho một hệ thống y tế cá nhân hóa, hiệu quả và thuận tiện hơn. Người bệnh có thể chủ động theo dõi tình trạng sức khỏe của mình mọi lúc, mọi nơi, giúp phát hiện sớm các dấu hiệu bất thường và đưa ra các biện pháp can thiệp kịp thời.

2.2.2 Nguyên lý hoạt động

Hệ thống IoT giám sát sức khỏe hoạt động dựa trên việc kết nối các thiết bị y tế thông minh với nhau thông qua mạng Internet. Các thiết bị này thu thập dữ liệu về các chỉ số sinh tồn của người bệnh như nhịp tim, huyết áp, nhịp thở, mức độ oxy trong máu, nhiệt độ cơ thể, v.v. Dữ liệu được truyền đến một nền tảng trung tâm (cloud platform) nơi chúng được xử lý, phân tích và lưu trữ. Quá trình hoạt động cụ thể có thể gồm các bước sau:

1. **Thu thập dữ liệu** Các cảm biến trên thiết bị đeo hoặc thiết bị y tế tại nhà sẽ liên tục đo và ghi lại các chỉ số sinh tồn của người bệnh. Dữ liệu thu thập được có thể bao gồm dữ liệu thời gian thực hoặc dữ liệu theo chu kỳ.
2. **Truyền dữ liệu** Dữ liệu được truyền từ các thiết bị đến nền tảng trung tâm thông qua các kết nối không dây như Wi-Fi, Bluetooth hoặc mạng di động. Các giao thức truyền thông như MQTT, CoAP thường được sử dụng để đảm bảo truyền dữ liệu hiệu quả và an toàn.
3. **Xử lý dữ liệu** Tại nền tảng trung tâm, dữ liệu được làm sạch, chuẩn hóa và phân tích để phát hiện các xu hướng, bất thường hoặc các sự kiện đáng chú ý. Các thuật toán học máy và trí tuệ nhân tạo có thể được áp dụng để xây dựng các mô hình dự đoán và phát hiện sớm các vấn đề sức khỏe.
4. **Hiển thị và cảnh báo** Dữ liệu được trực quan hóa trên một giao diện người dùng thân thiện, cho phép người dùng và các chuyên gia y tế theo dõi tình trạng sức khỏe của bệnh nhân. Hệ thống sẽ gửi các cảnh báo đến người dùng hoặc bác sĩ khi phát hiện các dấu hiệu bất thường hoặc khi cần can thiệp y tế.

2.3 Giới thiệu về phần mềm Adruino

2.3.1 Khái niệm

Arduino IDE (Integrated Development Environment) là một môi trường phát triển tích hợp, được thiết kế đặc biệt để tạo ra các chương trình (sketch) điều khiển các bo mạch Arduino. Với giao diện trực quan và dễ sử dụng, Arduino IDE cung cấp một nền tảng thuận tiện cho cả người mới bắt đầu và những người đã có kinh nghiệm trong lĩnh vực lập trình vi điều khiển [1].

Arduino IDE cho phép người dùng viết mã, biên dịch và tải chương trình lên bo mạch Arduino một cách nhanh chóng và hiệu quả. Ngôn ngữ lập trình chính được

sử dụng trong Arduino IDE là một biến thể của C++ được đơn giản hóa, giúp người dùng dễ dàng làm quen và nắm bắt.

2.3.2 Ứng dụng

Arduino IDE có vô vàn ứng dụng trong các lĩnh vực khác nhau, từ giáo dục, nghiên cứu đến ứng dụng thực tế. Nhờ tính linh hoạt và dễ sử dụng, Arduino IDE đã trở thành công cụ không thể thiếu cho những ai muốn khám phá và sáng tạo với điện tử. Một số ứng dụng điển hình của Arduino IDE bao gồm:

- Giáo dục: Arduino IDE được sử dụng rộng rãi trong các trường học, đại học để giảng dạy về lập trình vi điều khiển, điện tử và tự động hóa.
- Nghiên cứu: Arduino được sử dụng để xây dựng các nguyên mẫu cho các dự án nghiên cứu trong nhiều lĩnh vực khác nhau như y tế, nông nghiệp, môi trường.
- Đồ án cá nhân: Arduino là nền tảng tuyệt vời để thực hiện các dự án cá nhân như nhà thông minh, robot, hệ thống tự động hóa, thiết bị đo lường,...
- Sản xuất: Arduino được sử dụng để tạo ra các sản phẩm điện tử tùy chỉnh, các hệ thống điều khiển công nghiệp đơn giản.

Với cộng đồng người dùng lớn mạnh và thư viện hỗ trợ phong phú, Arduino IDE không ngừng mở rộng khả năng ứng dụng, tạo ra những sản phẩm và giải pháp sáng tạo.

2.3.3 Cấu trúc của một chương trình Arduino

Một chương trình Arduino, hay còn gọi là một "sketch", thường bao gồm hai hàm chính: setup() và loop(). Cấu trúc này tạo nên nền tảng cho hầu hết các chương trình Arduino.

Hàm setup(): là một phần không thể thiếu trong cấu trúc của một chương trình Arduino. Hàm này chỉ được thực thi một lần duy nhất khi chương trình bắt đầu chạy và có vai trò quan trọng trong việc khởi tạo và cấu hình ban đầu cho chương trình. Cụ thể, hàm setup() được sử dụng để khai báo và gán giá trị cho các biến, xác định các chân I/O (input/output) trên bo mạch, khởi tạo các thư viện cần thiết và thực hiện các cài đặt ban đầu khác để đảm bảo chương trình hoạt động đúng.

Hàm loop(): là trái tim của một chương trình Arduino. Sau khi hàm setup() hoàn thành việc khởi tạo các thiết lập ban đầu, hàm loop() sẽ được thực thi liên tục và lặp đi lặp lại vô hạn. Tất cả các lệnh mà bạn muốn chương trình thực hiện một cách tuần tự hoặc lặp đi lặp lại đều được đặt bên trong hàm loop() này. Đây là nơi diễn ra toàn bộ quá trình điều khiển và xử lý thông tin của chương trình, từ việc

đọc dữ liệu từ các cảm biến đến việc điều khiển các thiết bị đầu ra.

Thứ tự thực hiện của một chương trình Arduino luôn tuân theo quy tắc: hàm setup() được gọi trước, sau đó hàm loop() sẽ được thực thi liên tục trong một vòng lặp vô hạn. Hàm setup() có nhiệm vụ khởi tạo các thiết lập ban đầu cho chương trình, còn hàm loop() chứa toàn bộ logic điều khiển và các tác vụ lặp đi lặp lại. Ngôn ngữ lập trình được sử dụng trong Arduino là một biến thể của C++, được đơn giản hóa để người dùng dễ dàng tiếp cận và tạo ra các dự án của mình.

2.4 Giới thiệu về Arduino Uno R3

2.4.1 Tổng quan

Arduino Uno R3 là một trong những bo mạch vi điều khiển phổ biến nhất và được sử dụng rộng rãi trong cộng đồng người dùng Arduino trên toàn thế giới. Với thiết kế đơn giản, dễ sử dụng và khả năng mở rộng linh hoạt, Uno R3 là nền tảng lý tưởng cho những người mới bắt đầu làm quen với lập trình nhúng và các dự án điện tử [2].



Hình 2.1: Adruino Uno R3

2.4.2 Thông số kỹ thuật

Arduino Uno R3 là một bo mạch vi điều khiển đơn giản, dễ sử dụng và linh hoạt, được xây dựng dựa trên vi điều khiển ATmega328P của Atmel. Bo mạch này cung

cấp một nền tảng phát triển hoàn chỉnh cho các dự án điện tử, từ các ứng dụng đơn giản đến các hệ thống phức tạp [2].

Các thành phần chính:

- Vi điều khiển: ATmega328P (8-bit, tốc độ đồng hồ 16 MHz)
- Điện áp hoạt động: 5V
- Điện áp vào (khuyến nghị): 7-12V
- Dòng điện I/O trên mỗi chân: 40mA
- Bộ nhớ Flash: 32KB (0.5KB dùng cho bootloader)
- Bộ nhớ SRAM: 2KB
- Bộ nhớ EEPROM: 1KB
- Chân digital: 14 chân (trong đó có 6 chân PWM)
- Chân analog: 6 chân (độ phân giải 10 bit)
- Giao tiếp: UART, I2C, SPI
- Kích thước: 67mm x 53mm

Các thành phần chính trên bo mạch:

- Vi điều khiển ATmega328P: Trái tim của bo mạch, chịu trách nhiệm thực thi các lệnh của chương trình.
- Crystal oscillator: Cung cấp xung nhịp cho vi điều khiển.
- USB connector: Dùng để kết nối với máy tính, cấp nguồn và tải chương trình.
- Reset button: Nút dùng để khởi động lại bo mạch.
- Power jack: Cổng để cấp nguồn ngoài cho bo mạch.
- Chân digital: Dùng để đọc tín hiệu từ các cảm biến hoặc điều khiển các thiết bị ngoại vi.
- Chân analog: Dùng để đọc các tín hiệu tương tự như điện áp.

2.4.3 Quy trình hoạt động cơ bản

1. Viết chương trình: sử dụng Arduino IDE để viết chương trình bằng ngôn ngữ C/C++. Chương trình này sẽ mô tả các tác vụ mà bạn muốn Arduino thực hiện.
2. Tải chương trình: Chương trình được biên dịch thành mã máy và tải vào bộ nhớ flash của ATmega328P thông qua cổng USB.
3. Khởi động: Khi Arduino được cấp nguồn, chương trình sẽ bắt đầu thực thi từ

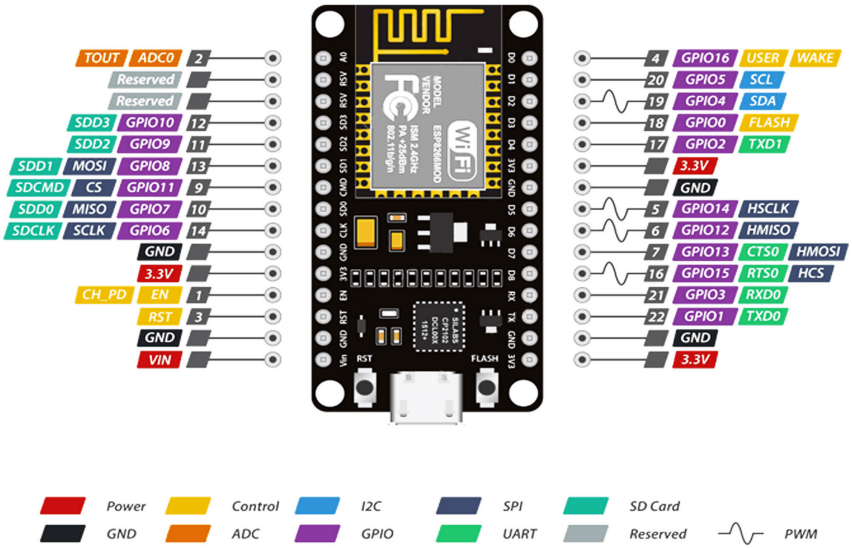
điểm bắt đầu.

4. Vòng lặp vô hạn: Hầu hết các chương trình Arduino đều có một vòng lặp vô hạn (loop()). Trong vòng lặp này, Arduino sẽ liên tục đọc dữ liệu từ các cảm biến, thực hiện các phép tính và điều khiển các thiết bị ngoại vi theo đúng các lệnh trong chương trình.

2.5 Giới thiệu về ESP8266 NodeMCU

2.5.1 Tổng quan

ESP8266 NodeMCU là một trong những bo mạch phát triển vi điều khiển không dây phổ biến nhất hiện nay, được xây dựng dựa trên chip Wi-Fi ESP8266 của Espressif Systems. Với kích thước nhỏ gọn, giá thành hợp lý và khả năng kết nối Internet, NodeMCU đã trở thành lựa chọn hàng đầu cho các dự án IoT (Internet of Things) và các ứng dụng cần đến kết nối mạng không dây [3].



Hình 2.2: Esp 8266 nodeMCU

NodeMCU không chỉ đơn thuần là một module Wi-Fi, mà còn là một nền tảng phát triển hoàn chỉnh với vi điều khiển, bộ nhớ, các chân I/O và các giao thức truyền thông cần thiết. Điều này giúp cho việc tạo ra các thiết bị IoT trở nên dễ dàng và nhanh chóng hơn bao giờ hết.

2.5.2 Thông số kỹ thuật

- Vi điều khiển: ESP8266EX (hoặc tương đương)
- Tần số CPU: 80 MHz
- Bộ nhớ Flash: 4MB (có thể thay đổi tùy phiên bản)
- Bộ nhớ SRAM: 64KB

- Điện áp I/O: 3.3V
- Dòng điện tiêu thụ: Tùy thuộc vào chế độ hoạt động và các thiết bị ngoại vi kết nối.

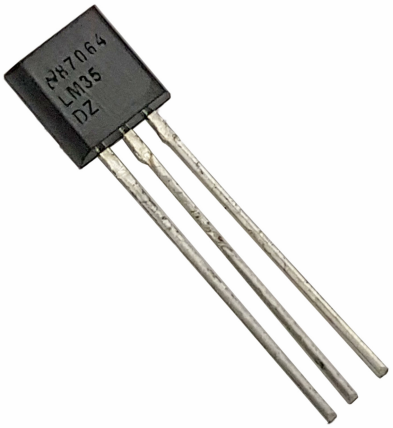
2.5.3 Quy trình hoạt động

1. Viết chương trình: sử dụng một môi trường phát triển tích hợp (IDE) như Arduino IDE, ESPHome hoặc các IDE chuyên dụng khác để viết mã. Mã này sẽ chứa các lệnh điều khiển các chức năng của NodeMCU, như kết nối Wi-Fi, đọc dữ liệu từ cảm biến, điều khiển các thiết bị ngoại vi.
2. Biên dịch: Chương trình bạn viết sẽ được biên dịch thành mã máy mà vi điều khiển ESP8266 có thể hiểu được.
3. Tải chương trình: Mã máy được tải vào bộ nhớ flash của NodeMCU thông qua cổng USB hoặc một giao thức truyền tải khác.
4. Khởi động: Khi NodeMCU được cấp nguồn, chương trình sẽ tự động bắt đầu chạy từ điểm bắt đầu.
5. Vòng lặp vô hạn: Chương trình sẽ thực hiện một vòng lặp vô hạn, liên tục kiểm tra các điều kiện, đọc dữ liệu từ các cảm biến và điều khiển các thiết bị ngoại vi theo các lệnh đã được lập trình.

2.6 Giới thiệu về cảm biến LM35

2.6.1 Tổng quan

Cảm biến LM35 là một linh kiện điện tử được sử dụng rộng rãi trong các hệ thống đo lường và điều khiển nhiệt độ. Với cấu trúc đơn giản, độ chính xác cao và giá thành hợp lý, LM35 trở thành lựa chọn hàng đầu cho nhiều ứng dụng khác nhau.



Hình 2.3: LM35

LM35 hoạt động dựa trên nguyên lý biến đổi nhiệt độ thành tín hiệu điện áp. Cụ thể, điện áp đầu ra của cảm biến tỉ lệ thuận với nhiệt độ đo được, với hệ số chuyển đổi là 10mV/°C. Điều này có nghĩa là cứ mỗi độ tăng lên của nhiệt độ, điện áp đầu ra sẽ tăng thêm 10mV. Nhờ đặc tính tuyến tính này, việc xử lý và hiển thị dữ liệu nhiệt độ trở nên dễ dàng hơn [4].

2.6.2 Thông số kỹ thuật

- Điện áp hoạt động (Vcc): 4V - 20V
- Dòng tiêu thụ (Icc): 60 A
- Độ phân giải: 10 mV/°C
- Độ chính xác: ±0.25°C
- Dải nhiệt độ hoạt động: -55°C - 150°C
- Trở kháng đầu ra: 0.1

2.6.3 Quy trình hoạt động

Cảm biến LM35 hoạt động dựa trên nguyên lý biến đổi nhiệt năng thành điện năng. Bên trong cảm biến, có một vật liệu bán dẫn có điện trở thay đổi theo nhiệt độ. Khi nhiệt độ tăng lên, điện trở của vật liệu này giảm đi và ngược lại.

Quy trình hoạt động cụ thể như sau:

1. Cung cấp nguồn điện: Cảm biến LM35 được cấp nguồn điện một chiều (DC) thông qua hai chân Vcc và GND.
2. Đo nhiệt độ: Vật liệu bán dẫn bên trong cảm biến tiếp xúc trực tiếp với môi

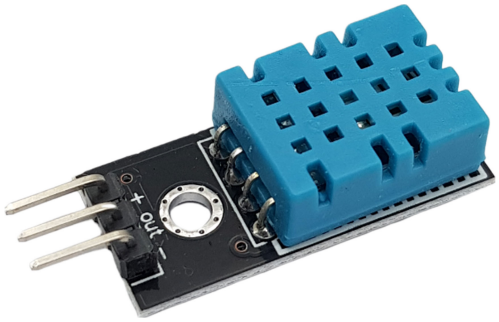
trường cần đo. Nhiệt độ của môi trường sẽ làm thay đổi điện trở của vật liệu này.

3. Tạo ra điện áp tỉ lệ: Sự thay đổi điện trở của vật liệu bán dẫn sẽ dẫn đến sự thay đổi dòng điện chạy qua nó. Dòng điện này đi qua một điện trở cố định bên trong cảm biến, tạo ra một điện áp tỉ lệ thuận với nhiệt độ. Điện áp này được xuất ra ở chân output (Vout) của cảm biến.
4. Đọc giá trị: Điện áp Vout được đưa vào mạch xử lý để chuyển đổi thành giá trị nhiệt độ tương ứng. Giá trị này có thể được hiển thị trên màn hình, lưu trữ trong bộ nhớ hoặc sử dụng để điều khiển các thiết bị khác.

2.7 Giới thiệu về DHT11

2.7.1 Tổng quan

Cảm biến DHT11 là một linh kiện điện tử được sử dụng rộng rãi trong các dự án điện tử, IoT và tự động hóa. Với khả năng đo đồng thời nhiệt độ và độ ẩm tương đối, DHT11 cung cấp một giải pháp đơn giản và hiệu quả để giám sát điều kiện môi trường. Cảm biến này có cấu trúc nhỏ gọn, dễ dàng tích hợp vào các thiết bị khác nhau và đặc biệt phù hợp với những dự án có yêu cầu về chi phí thấp [5].



Hình 2.4: DHT11

2.7.2 Thông số kỹ thuật

- Điện áp hoạt động (Vcc) 3.3 - 5.5V
- Dòng tiêu thụ (Icc) 1.0mA
- Độ phân giải nhiệt độ 0.1°C
- Phạm vi đo nhiệt độ 0 - 50°C

- Độ chính xác nhiệt độ ±2°C
- Độ phân giải độ ẩm 5%RH
- Phạm vi đo độ ẩm 20 - 90%RH
- Độ chính xác độ ẩm ±5%RH
- Thời gian đáp ứng 250 ms
- Giao tiếp 1-Wire

2.7.3 Quy trình hoạt động

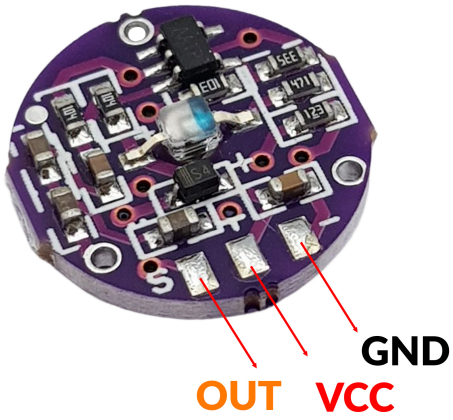
Cảm biến DHT11 hoạt động dựa trên nguyên lý đo điện trở của một chất bán dẫn nhạy cảm với độ ẩm và sử dụng một thermistor (điện trở nhiệt) để đo nhiệt độ. Quy trình hoạt động của DHT11 có thể được chia thành các bước sau:

1. Cung cấp nguồn: Cảm biến được kết nối với nguồn điện một chiều (DC) trong khoảng 3.3V đến 5.5V. Sau đó vi điều khiển gửi một xung tín hiệu thấp kéo dài ít nhất 18ms để khởi động quá trình truyền dữ liệu.
2. Sau khi nhận được tín hiệu bắt đầu, DHT11 sẽ trả lời bằng một xung thấp kéo dài 20-21us để xác nhận đã nhận được lệnh. Tiếp theo, DHT11 sẽ giữ trạng thái cao trong khoảng 26-28us.
3. DHT11 sẽ truyền 40 bit dữ liệu, tương ứng với 5 byte dữ liệu. Mỗi byte gồm 8 bit, trong đó bit đầu tiên là bit kiểm tra, các bit còn lại là dữ liệu nhiệt độ và độ ẩm. Mỗi bit trong dữ liệu được biểu diễn bằng một xung điện. Nếu là bit 0, xung sẽ kéo dài 50 micro giây, còn nếu là bit 1, xung sẽ kéo dài từ 26 đến 28 micro giây. Vi điều khiển sẽ đọc các xung này và dịch mã thành các giá trị nhiệt độ và độ ẩm tương ứng.
4. Vi điều khiển sẽ đọc từng bit dữ liệu truyền từ DHT11 và lưu trữ vào bộ nhớ. Sau đó thực hiện các phép tính để chuyển đổi dữ liệu nhị phân thành giá trị nhiệt độ và độ ẩm thực tế. Dữ liệu nhiệt độ và độ ẩm thu được có thể được sử dụng để hiển thị trên màn hình, lưu trữ hoặc điều khiển các thiết bị khác.

2.8 Giới thiệu về Pulse Sensor

2.8.1 Tổng quan

Cảm biến nhịp tim Pulse Sensor là một module điện tử được thiết kế đặc biệt để đo nhịp tim của con người một cách đơn giản và hiệu quả. Với kích thước nhỏ gọn, dễ sử dụng và giá thành hợp lý, Pulse Sensor đã trở thành một lựa chọn phổ biến trong các dự án điện tử, đặc biệt là các dự án liên quan đến sức khỏe và theo dõi sinh trắc học [6].



Hình 2.5: Pulse Sensor

2.8.2 Thông số kỹ thuật

- Điện áp hoạt động: 3.3V - 5V
- Dòng tiêu thụ: < 4mA (tại 5V)
- Ngõ ra: Tín hiệu analog, 0V - 5V
- Tần số lấy mẫu: 100Hz
- Kích thước: 25mm x 15mm x 5mm
- Trọng lượng: 2g
- Chất liệu: nhựa ABS, linh kiện điện tử
- Màu sắc: Đỏ
- Nhiệt độ hoạt động: 0°C - 50°C
- Độ ẩm hoạt động: 20% - 80%RH
- Giao tiếp: Analog
- Độ chính xác: ± 5%
- Độ phân giải: 0.1 bpm
- Thời gian đáp ứng: 100ms

2.8.3 Quy trình hoạt động

1. Cung cấp nguồn điện: Cảm biến được kết nối với nguồn điện có điện áp phù hợp (3.3V - 5V).
2. Phát ánh sáng: Các LED phát ra ánh sáng với bước sóng thích hợp (thường là

ánh sáng đỏ hoặc hồng ngoại) chiếu vào mô mềm.

3. Thu nhận ánh sáng phản xạ: Một photodiode thu nhận ánh sáng phản xạ từ mô.
4. Chuyển đổi thành tín hiệu điện: Ánh sáng phản xạ được chuyển đổi thành tín hiệu điện tương ứng với lượng máu chảy qua.
5. Xử lý tín hiệu: Tín hiệu điện thu được sẽ được xử lý để lọc nhiễu, khuếch đại và chuyển đổi thành dạng số.
6. Tính toán nhịp tim: Dựa trên các đỉnh và đáy của tín hiệu, mạch xử lý sẽ tính toán khoảng thời gian giữa hai nhịp tim liên tiếp và từ đó xác định nhịp tim.
7. Đưa ra kết quả: Kết quả đo nhịp tim có thể được hiển thị trên một màn hình, truyền đến một thiết bị khác hoặc lưu trữ lại.

CHƯƠNG 3. PHÂN TÍCH YÊU CẦU

3.1 Mục đích

Mục đích chính của hệ thống:

- **Giám sát liên tục các chỉ số sinh tồn:**
  - **Nhịp tim:** Theo dõi nhịp tim để phát hiện các bất thường như nhịp tim nhanh, chậm hoặc không đều, có thể là dấu hiệu của các vấn đề tim mạch.
  - **Nhiệt độ:** Đo nhiệt độ cơ thể qua đầu ngón tay để phát hiện sốt, hạ thân nhiệt hoặc các thay đổi bất thường khác, có thể báo hiệu nhiễm trùng hoặc các vấn đề sức khỏe khác.
  - **Độ ẩm và nhiệt độ phòng:** Đo độ ẩm và nhiệt độ trong phòng để đảm bảo môi trường phòng bệnh phù hợp.
    * Độ ẩm quá thấp có thể gây khô da, niêm mạc, dễ dẫn đến các vấn đề về hô hấp, đặc biệt ở người bệnh mãn tính hoặc người cao tuổi.
    * Độ ẩm quá cao lại tạo điều kiện thuận lợi cho vi khuẩn, nấm mốc phát triển, gây hại cho sức khỏe.
    * Nhiệt độ phòng nếu quá thấp hoặc quá cao có thể khiến cho thân nhiệt của người bệnh bị ảnh hưởng, dẫn đến sốt
- **Phát hiện sớm các dấu hiệu bệnh tật:** Phân tích dữ liệu cảm biến để phát hiện các dấu hiệu sớm của các bệnh lý như tim mạch, hô hấp, nhiễm trùng, hoặc các biến chứng của bệnh mãn tính.
- **Cảnh báo kịp thời**: Gửi thông báo đến người bệnh, người nhà hoặc nhân viên y tế khi phát hiện các bất thường hoặc khi các chỉ số vượt quá ngưỡng cảnh báo.
- **Hỗ trợ quá trình điều trị:** Cung cấp dữ liệu chi tiết về tình trạng sức khỏe của bệnh nhân cho bác sĩ để đánh giá hiệu quả điều trị, điều chỉnh phác đồ điều trị và đưa ra các quyết định lâm sàng kịp thời.
- **Tăng cường sự tự chủ của bệnh nhân:** Giúp bệnh nhân theo dõi tình trạng sức khỏe của mình tại nhà, tăng cường sự tự tin và chủ động trong việc quản lý bệnh.
- **Giảm tải cho hệ thống y tế:** Giảm số lượng bệnh nhân đến khám tại bệnh viện, giảm tải cho nhân viên y tế và tiết kiệm chi phí.

3.2 Phạm vi hệ thống

Đối với nhân viên y tế

- Quản lý hồ sơ điện tử bệnh nhân
- Quản lý dữ liệu
- Quản lý hệ thống
- Quản lý cảnh báo
- Theo dõi tình trạng sức khỏe của nhiều bệnh nhân cùng lúc.
- Phân tích dữ liệu để đưa ra quyết định điều trị.

Đối với người bệnh

- Theo dõi trực tiếp tình trạng sức khỏe của mình.
- Nhận được cảnh báo kịp thời khi có vấn đề xảy ra.

3.3 Yêu cầu hệ thống

Đối với quản trị viên nhân viên y tế

1. **Quản lý người dùng**
    - Cấp quyền truy cập cho từng người dùng (bệnh nhân, nhân viên y tế).
2. **Quản lý dữ liệu:**
    - Truy cập và xem toàn bộ dữ liệu của hệ thống.
    - Xuất dữ liệu dưới nhiều định dạng khác nhau.
    - Xóa dữ liệu khi cần thiết (tuân thủ quy định về bảo mật dữ liệu).
3. **Quản lý cảnh báo**
    - Thiết lập các ngưỡng cảnh báo.
    - Cấu hình thông báo (thông báo trên ứng dụng).
    - Kiểm tra và xử lý các thông báo.
4. **Truy cập và quản lý thông tin bệnh nhân** Theo dõi các chỉ số cơ thể bệnh nhân. Quan sát trực quan các biểu đồ thể hiện sự thay đổi của nhịp tim, nhiệt độ, độ ẩm và các chỉ số khác theo thời gian.
5. **Cảnh báo và thông báo** Nhận được các thông báo tức thời khi các chỉ số sinh tồn của bệnh nhân vượt quá ngưỡng cảnh báo hoặc có sự thay đổi bất thường.

Đốivới bệnh nhân

1. **Theo dõi sức khỏe cá nhân**

- Xem lại các chỉ số sinh tồn đã đo được trong quá khứ bao và hiện tại gồm: nhịp tim, nhiệt độ và độ ẩm.
- Nhận cảnh báo về sức khỏe nếu có điều gì bất thường xảy ra.

3.4 Yêu cầu chi tiết theo nhóm người dùng

3.4.1 Quản trị viên nhân viên y tế

- **Front-end**

1. Bảng điều khiển tổng quan: Hiển thị thông tin hệ thống, số lượng người dùng, thiết bị, cảnh báo.
2. Module quản lý người dùng: Thêm, sửa, xóa, phân quyền, tìm kiếm người dùng.
3. Module quản lý cảnh báo: Thiết lập ngưỡng cảnh báo, xem lịch sử cảnh báo.
4. Xem thông tin bệnh nhân, lịch sử sức khỏe, các chỉ số sinh tồn theo thời gian thực.
5. Lọc, tìm kiếm bệnh nhân, xem chi tiết hồ sơ sức khỏe.
6. Xem danh sách cảnh báo, chi tiết cảnh báo, xác nhận xử lý cảnh báo.
7. Gửi thông báo đến nhân viên y tế khi có cảnh báo.

- **Back-end**

1. API quản lý người dùng, thiết bị, dữ liệu, cảnh báo.
2. Cơ sở dữ liệu lưu trữ thông tin người dùng, thiết bị, dữ liệu sinh trắc, lịch sử cảnh báo.
3. Xem danh sách thiết bị, cấu hình, cập nhật firmware, xem lịch sử sự cố.
4. Thiết lập ngưỡng cảnh báo, xem lịch sử cảnh báo.

3.4.2 Bệnh nhân

- **Front-end**

1. Xem thông tin sức khỏe cá nhân, lịch sử đo, các chỉ số sinh tồn.

- **Back-end**

1. Ứng dụng web kết nối với API để lấy dữ liệu cá nhân.

3.4.3 Thiết bị

- **Cảm biến nhịp tim**

1. Đo nhịp tim trong khoảng 20-200 nhịp/phút.

2. Nếu nhịp tim ngoài khoảng này, báo lỗi thiết bị.
3. Có ngưỡng cảnh báo nếu nhỏ hơn 60 nhịp/phút hoặc lớn hơn 120 nhịp/phút tuy nhiên có thể thay đổi bởi nhân viên y tế.

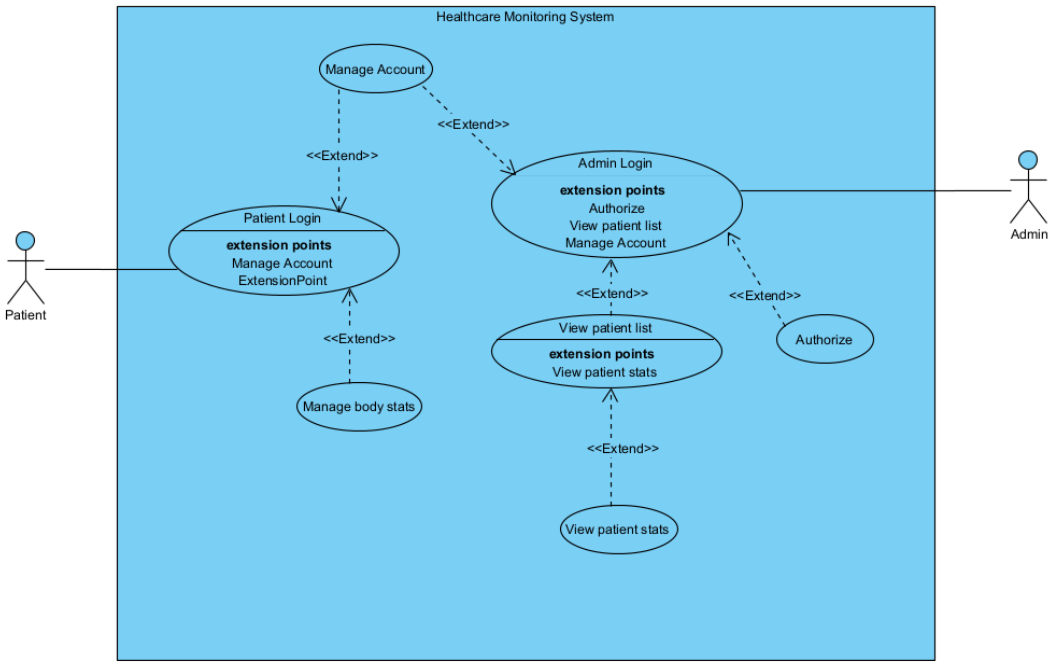
- **Cảm biến nhiệt độ**

1. Đo nhiệt độ trong khoảng từ 20°C đến 50°C.
2. Nếu nhiệt độ ngoài khoảng này, báo lỗi thiết bị.
3. Có ngưỡng cảnh báo nếu nhỏ hơn 35°C hoặc lớn hơn 39°C tuy nhiên có thể thay đổi bởi nhân viên y tế.

- **Cảm biến DHT11**

1. Đo độ ẩm trong khoảng từ 20% đến 90% RH (Relative Humidity).
2. Nếu độ ẩm ngoài khoảng này, báo lỗi thiết bị.
3. Đo nhiệt độ trong khoảng từ 20°C đến 50°C.
4. Nếu nhiệt độ ngoài khoảng này, báo lỗi thiết bị.
5. Có ngưỡng cảnh báo nếu nhỏ hơn 15°C hoặc lớn hơn 35°C tuy nhiên có thể thay đổi bởi nhân viên y tế.

3.5 Sơ đồ Use Case tổng quát



Hình 3.1: Sơ đồ Use Case Tổng Quát

Hệ thống bao gồm:

- Sáu usecase cho bệnh nhân:
  - Bệnh nhân register
  - Bệnh nhân login
  - Xem chi tiết thông tin cá nhân bệnh nhân
  - Xem các thông số đo chỉ số bệnh nhân và phòng bệnh nhân trực tiếp
  - Xem lịch sử các thông số đo chỉ số bệnh nhân và phòng bệnh nhân
  - Cảnh báo nguy hiểm khi chỉ số vượt ngưỡng
- Năm usecase cho quản trị viên nhân viên y tế:
  - Quản trị viên nhân viên y tế login
  - Xem danh sách các bệnh nhân
  - Xem chi tiết chỉ số sinh tồn của một bệnh nhân
  - Cài đặt ngưỡng cảnh báo
  - Cảnh báo nguy hiểm khi chỉ số vượt ngưỡng

CHƯƠNG 4. THIẾT KẾ HỆ THỐNG

4.1 Mục tiêu hệ thống

Mục tiêu chính hệ thống bao gồm:

- **Thu thập dữ liệu sinh trắc học liên tục:** Hệ thống sẽ sử dụng các cảm biến chuyên dụng (như cảm biến đo nhịp tim, cảm biến nhiệt độ) gắn trực tiếp lên ngón tay dùng để đo nhịp tim và nhiệt độ một cách liên tục. Tần suất thu thập dữ liệu có thể được tùy chỉnh (ví dụ: mỗi giây, mỗi phút) để đáp ứng yêu cầu theo dõi của từng đối tượng.
- **Xử lý và lưu trữ dữ liệu:** Dữ liệu thu thập được sẽ được truyền đến một nền tảng đám mây an toàn để lưu trữ và xử lý.
- **Cung cấp giao diện người dùng trực quan:** Bác sĩ sẽ có một bảng điều khiển trực quan, hiển thị các biểu đồ, đồ thị thể hiện sự thay đổi của nhịp tim và nhiệt độ theo thời gian, giúp dễ dàng theo dõi tiến triển của bệnh nhân. Hệ thống sẽ gửi cảnh báo tự động đến bác sĩ khi phát hiện các dấu hiệu bất thường hoặc khi các chỉ số vượt quá ngưỡng cho phép.

4.2 Hành vi và các hoạt động của hệ thống

Thu thập dữ liệu

- Đo đạc liên tục: Hệ thống sẽ thực hiện việc đo nhịp tim và nhiệt độ một cách liên tục thông qua các cảm biến đặt trên ngón tay người dùng.
- Người dùng hoặc bác sĩ có thể điều chỉnh tần suất thu thập dữ liệu để phù hợp với nhu cầu theo dõi, từ đo mỗi giây cho đến mỗi vài phút.
- Lưu trữ dữ liệu tạm thời: Dữ liệu đo được sẽ được lưu trữ tạm thời trong bộ nhớ của thiết bị IoT trước khi được truyền lên đám mây.

Truyền dữ liệu

- Kết nối không dây: Thiết bị IoT sẽ sử dụng kết nối Wi-Fi để truyền dữ liệu về nền tảng đám mây.
- Định dạng dữ liệu: Dữ liệu được truyền đi sẽ được mã hóa theo một định dạng nhất định để đảm bảo tính bảo mật và dễ dàng xử lý.
- Lưu trữ dữ liệu tạm thời: Dữ liệu đo được sẽ được lưu trữ tạm thời trong bộ nhớ của thiết bị IoT trước khi được truyền lên đám mây.

Bảo mật dữ liệu

- Mã hóa dữ liệu: Tất cả dữ liệu được truyền đi và lưu trữ đều phải được mã hóa

để bảo vệ thông tin khỏi bị truy cập trái phép.

- Xác thực người dùng: Thực hiện xác thực người dùng để tăng cường bảo mật.

Giao diện người dùng

- Hiển thị dữ liệu trực quan: Bảng điều khiển của bác sĩ sẽ hiển thị các biểu đồ, đồ thị thể hiện sự thay đổi của nhịp tim và nhiệt độ theo thời gian, giúp dễ dàng theo dõi tình trạng sức khỏe của bệnh nhân.
- Thiết lập ngưỡng cảnh báo: Bác sĩ hoặc người dùng có thể thiết lập các ngưỡng cảnh báo cho các chỉ số sinh trắc học.

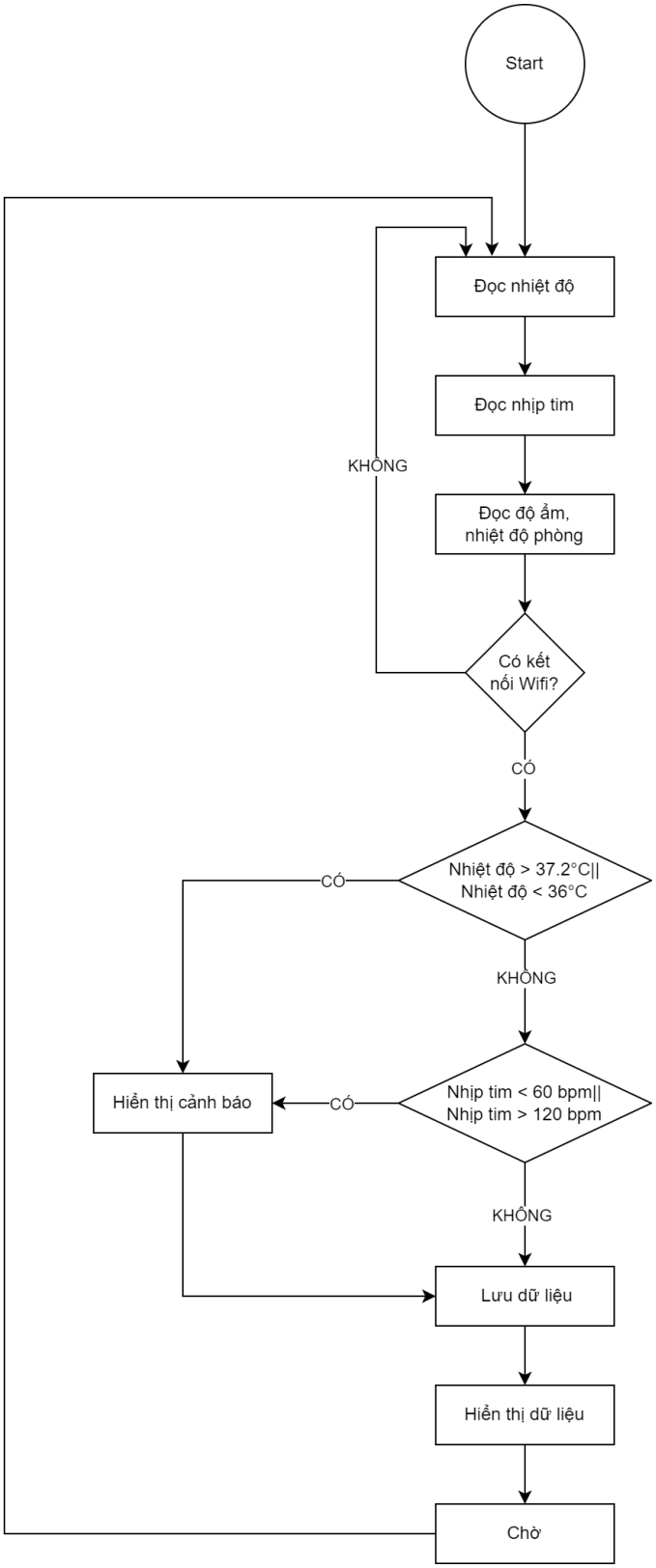
Quản lý hệ thống

- Sao lưu dữ liệu: Thực hiện sao lưu dữ liệu định kỳ để phòng ngừa trường hợp mất dữ liệu do sự cố kỹ thuật.
- Quản lý người dùng: Thiết lập hệ thống quản lý người dùng với các quyền truy cập khác nhau để đảm bảo tính bảo mật.
- Bảo trì thiết bị: Thực hiện bảo trì định kỳ cho các thiết bị IoT để đảm bảo chúng hoạt động ổn định.

Quyền riêng tư

- Thông báo rõ ràng: Người dùng cần được thông báo rõ ràng về mục đích thu thập và sử dụng dữ liệu, các loại dữ liệu được thu thập và cách thức bảo vệ dữ liệu.
- Quyền truy cập và kiểm soát: Người dùng có quyền truy cập và kiểm soát dữ liệu cá nhân của mình, bao gồm quyền xem, sửa đổi và xóa dữ liệu.
- Minh bạch: Quá trình thu thập, xử lý và lưu trữ dữ liệu phải được thực hiện một cách minh bạch và có thể kiểm soát được.

4.3 Đặc tả tiến trình



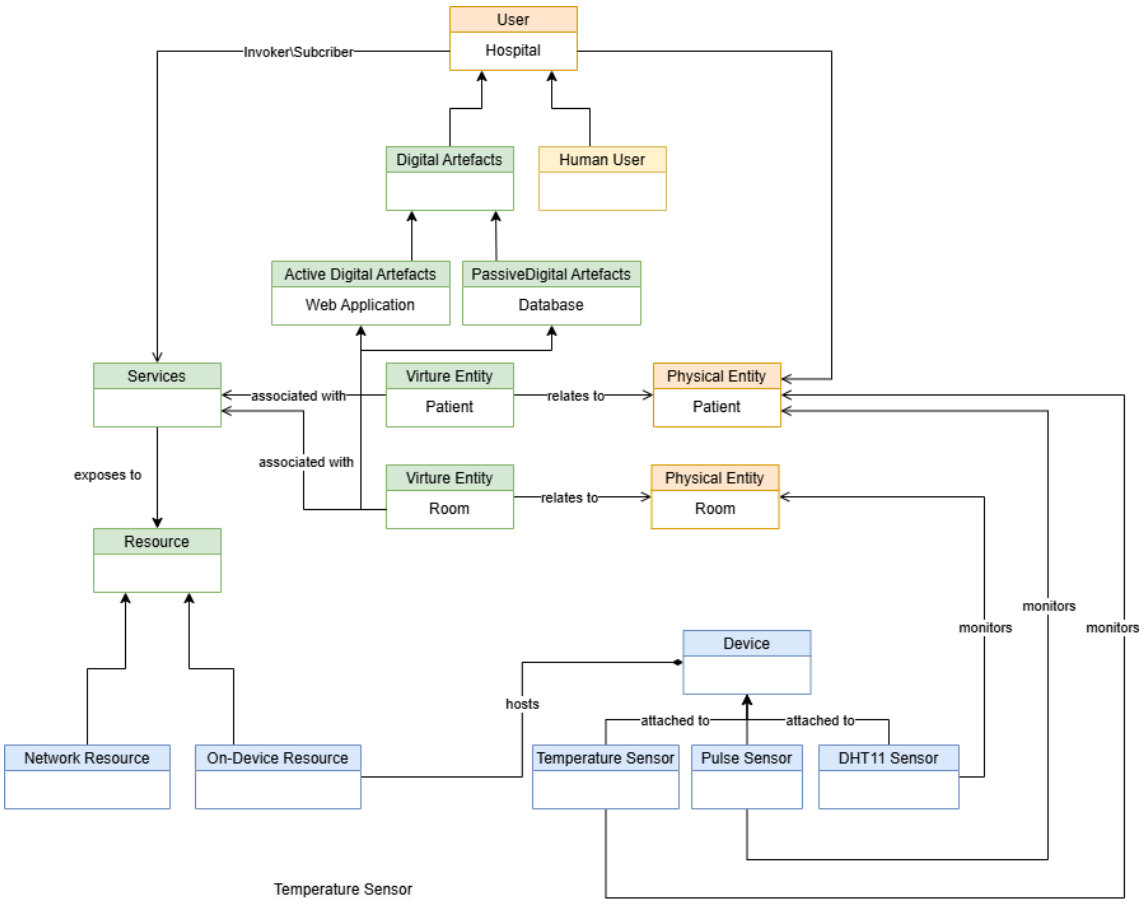
Hình 4.1: Đặc tả tiến trình

Mô tả tiến trình hệ thống:

- Bước 1: Khởi động hệ thống. Hệ thống bắt đầu hoạt động.

- Bước 2: Đọc dữ liệu cảm biến. Các cảm biến tiến hành đo nhiệt độ, nhịp tim người, nhiệt độ và độ ẩm phòng.
- Bước 3: Kiểm tra kết nối Wifi. Hệ thống xác minh kết nối Wifi. Nếu không có kết nối, quay lại bước 2. Nếu có, tiếp tục bước 4.
- Bước 4: Kiểm tra ngưỡng cảnh báo. Hệ thống kiểm tra các chỉ số đo được có vượt ngưỡng cảnh báo không. Nếu có, chuyển sang bước 4.1. Nếu không, tiếp tục bước 5.
- Bước 4.1: Hiển thị cảnh báo. Hệ thống hiển thị cảnh báo nguy hiểm nếu các chỉ số vượt ngưỡng.
- Bước 5: Lưu trữ dữ liệu. Dữ liệu được lưu vào cơ sở dữ liệu.
- Bước 6: Hiển thị dữ liệu. Dữ liệu được hiển thị lên trang web.
- Bước 7: Chờ và lặp lại. Hệ thống chờ một khoảng thời gian đã định trước và lặp lại từ bước 2.

4.4 Đặc tả mô hình miền



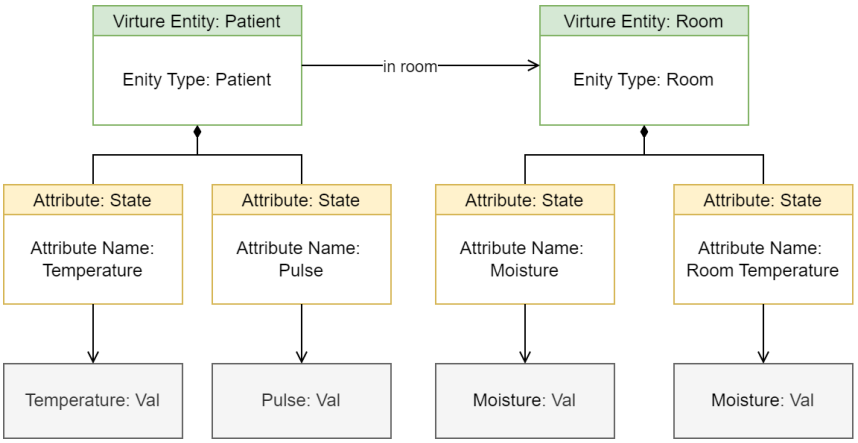
Hình 4.2: Đặc tả mô hình miền

Hệ thống có các miền chính như sau:

- Miền người dùng (User Domain):
  - Hospital: Đây là nơi cung cấp dịch vụ y tế và chăm sóc sức khỏe cho bệnh nhân. Bệnh viện có thể bao gồm nhiều phòng ban và khu vực chức năng khác nhau.
  - Human User: Bao gồm các bệnh nhân sử dụng dịch vụ y tế. Họ là những người trực tiếp nhận các dịch vụ chăm sóc sức khỏe và theo dõi tình trạng sức khỏe của mình.
- Miền vật lý (Physical Domain):
  - Bệnh nhân (Patient): Là những người cần được theo dõi và chăm sóc sức khỏe. Họ có thể ở trong bệnh viện hoặc tại nhà.
  - Phòng (Room): Các phòng trong bệnh viện hoặc tại nhà bệnh nhân, nơi các thiết bị cảm biến được lắp đặt để theo dõi các chỉ số sức khỏe và môi trường.
- Miền ảo (Virtual Entity)
  - Bệnh nhân (Patient): Đại diện ảo của bệnh nhân trong hệ thống, giúp theo dõi và quản lý thông tin sức khỏe.
  - Phòng (Room): Đại diện ảo của các phòng, giúp quản lý và giám sát các điều kiện môi trường trong phòng.
- Miền thiết bị (Device Domain):
  - Cảm biến nhiệt độ (Temperature Sensor): Đo nhiệt độ cơ thể bệnh nhân.
  - Cảm biến nhịp tim (Pulse Sensor): Đo nhịp tim của bệnh nhân.
  - Cảm biến nhiệt độ và độ ẩm (DHT11 Sensor): Đo nhiệt độ và độ ẩm của phòng, giúp đảm bảo môi trường phù hợp cho bệnh nhân.
- Miền tài nguyên (Resource Domain):
  - Tài nguyên mạng (Network Resources): Bao gồm các kết nối mạng cần thiết để truyền dữ liệu từ các cảm biến đến hệ thống quản lý.
  - Tài nguyên phần cứng thiết bị (On-Device Resources): Bao gồm các thiết bị phần cứng như máy chủ, bộ lưu trữ và các thiết bị mạng.
- Miền dịch vụ (Service Domain):
  - Ứng dụng Web (Web Application): Giao diện người dùng để truy cập và quản lý dữ liệu sức khỏe, cung cấp thông tin và cảnh báo cho người dùng.
  - Cơ sở dữ liệu (Database): Lưu trữ tất cả dữ liệu thu thập từ các cảm biến

và thông tin người dùng, đảm bảo dữ liệu được quản lý và truy xuất hiệu quả.

4.5 Đặc tả mô hình thông tin



Hình 4.3: Đặc tả mô hình thông tin

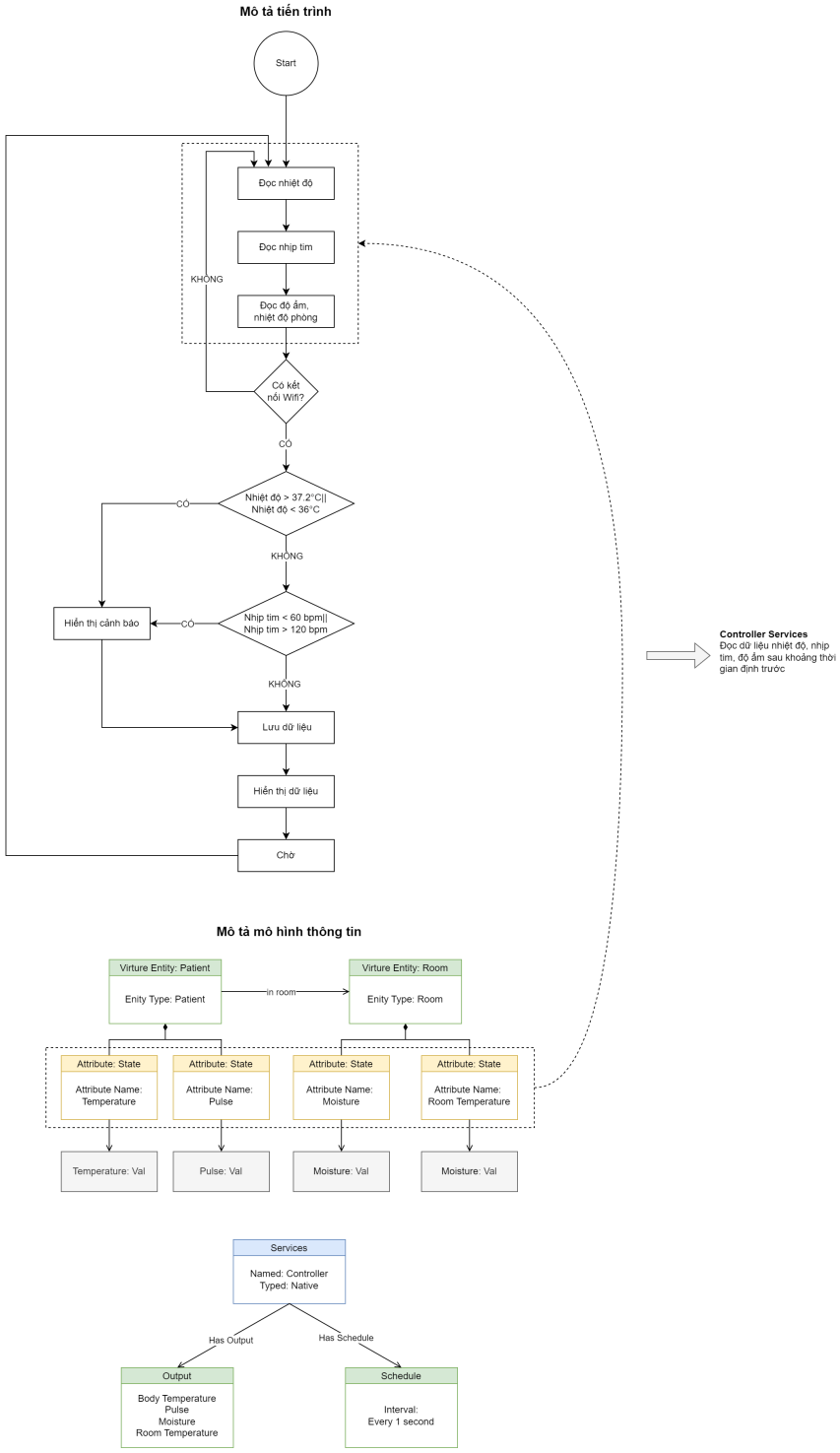
Mô hình thông tin

Mỗi thực thể ảo trong hệ thống đều có những đặc trưng riêng biệt, giúp theo dõi và quản lý thông tin một cách chính xác và hiệu quả.

1. Patient
    - Nhiệt độ cơ thể (Temperature): Đây là một giá trị thực, đo lường nhiệt độ của cơ thể bệnh nhân. Việc theo dõi nhiệt độ cơ thể giúp phát hiện sớm các dấu hiệu bất thường như sốt, hạ thân nhiệt, từ đó có thể can thiệp kịp thời.
    - Nhịp tim (Pulse): Cũng là một giá trị thực, nhịp tim đo lường số lần tim đập trong một phút. Theo dõi nhịp tim giúp đánh giá tình trạng sức khỏe tim mạch của bệnh nhân, phát hiện các vấn đề như nhịp tim không đều, nhịp tim quá nhanh hoặc quá chậm.
2. Room
    - Nhiệt độ phòng (Room Temperature): Đây là giá trị thực, đo lường nhiệt độ trong phòng nơi bệnh nhân đang ở. Nhiệt độ phòng cần được duy trì ở mức phù hợp để đảm bảo sự thoải mái và an toàn cho bệnh nhân, đặc biệt là trong các trường hợp bệnh nhân có yêu cầu đặc biệt về môi trường sống.
    - Độ ẩm phòng (Room Moisture): Cũng là một giá trị thực, đo lường mức độ ẩm trong không khí của phòng. Độ ẩm phòng cần được kiểm soát để tránh các vấn đề về hô hấp, da khô, và các bệnh liên quan đến môi trường

ẩm thấp hoặc quá khô.

4.6 Thông số dịch vụ



Hình 4.4: Thông số dịch vụ

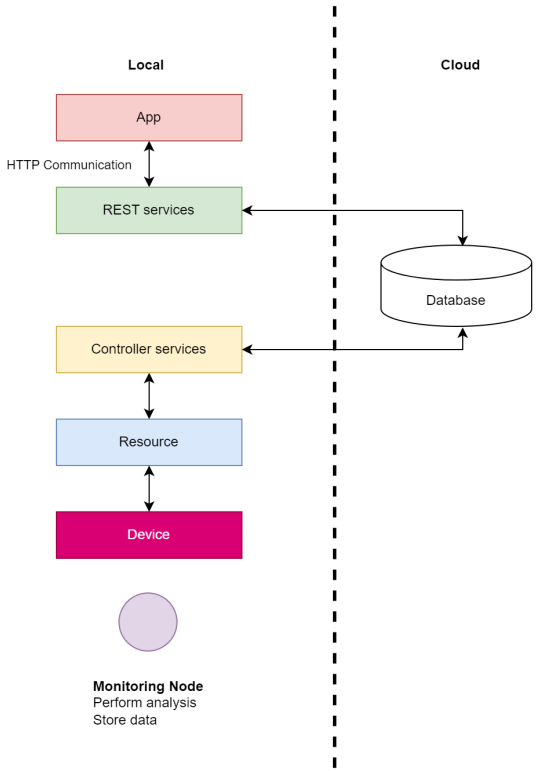
Miêu tả thông số dịch vụ

Dịch vụ Controller đóng vai trò quan trọng trong hệ thống, với nhiệm vụ chính là điều khiển quá trình đọc dữ liệu từ các cảm biến. Dịch vụ này đảm bảo rằng dữ liệu được thu thập một cách chính xác và kịp thời, từ đó cung cấp thông tin cần thiết

cho các thành phần khác của hệ thống để phân tích và xử lý. Chức năng chính của dịch vụ Controller:

- Điều khiển cảm biến: Dịch vụ Controller quản lý và điều khiển hoạt động của các cảm biến, bao gồm cảm biến nhiệt độ, cảm biến nhịp tim, và cảm biến nhiệt độ và độ ẩm. Nó đảm bảo rằng các cảm biến hoạt động đúng cách và thu thập dữ liệu liên tục.
- Thu thập dữ liệu: Dịch vụ này chịu trách nhiệm thu thập dữ liệu từ các cảm biến theo các khoảng thời gian đã định trước. Dữ liệu thu thập bao gồm các thông số quan trọng như nhiệt độ cơ thể, nhịp tim của bệnh nhân, nhiệt độ và độ ẩm phòng.
- Truyền dữ liệu: Dữ liệu sau khi được xử lý sẽ được truyền tới các thành phần khác của hệ thống, như cơ sở dữ liệu và ứng dụng web, để lưu trữ và hiển thị. Dịch vụ Controller đảm bảo rằng quá trình truyền dữ liệu diễn ra suôn sẻ và không bị gián đoạn.

4.7 Đặc tả cấp độ IoT



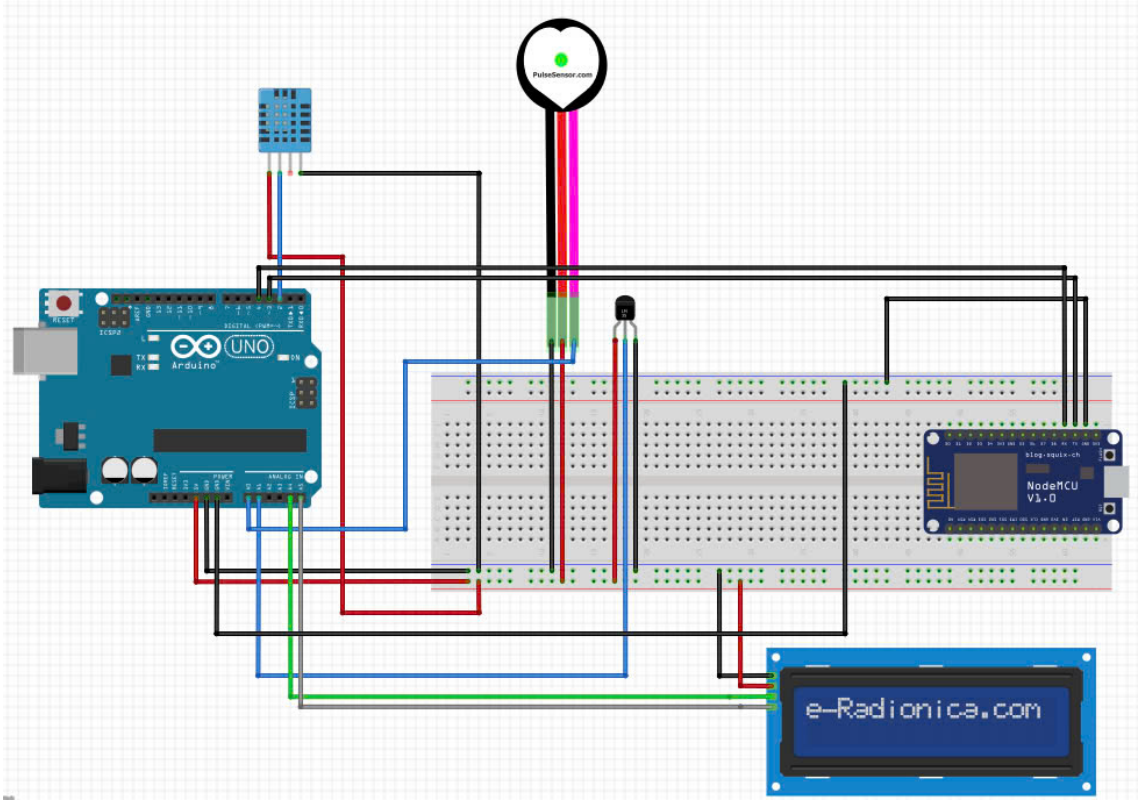
Hình 4.5: Đặc tả cấp độ IoT

Đặc tả cấp độ IoT Trong sơ đồ trên, ứng web giao tiếp với dịch vụ RESTful thông qua các yêu cầu HTTP. Dịch vụ RESTful sẽ chuyển tiếp các yêu cầu này đến dịch vụ điều khiển hoặc truy vấn cơ sở dữ liệu cloud. Dữ liệu từ các cảm biến được

gửi đến dịch vụ điều khiển thông qua giao thức MQTT. Dịch vụ điều khiển sẽ xử lý dữ liệu và gửi lệnh điều khiển đến các thiết bị.

CHƯƠNG 5. PHÂN TÍCH HỆ THỐNG

5.1 Kết nối thiết bị



Hình 5.1: Sơ đồ kết nối thiết bị

5.2 Danh sách công việc cần thực hiện

Code project xem chi tiết tại [7]

5.2.1 Đọc và xử lý thông tin từ các sensor

- **Setup** (arduino_main.ino)
  - Hàm setup khởi tạo các thành phần khác nhau được sử dụng trong dự án. Nó thiết lập cảm biến nhịp tim bằng cách cấu hình đầu vào analog và ngưỡng, khởi tạo giao tiếp serial cho cả Arduino và module ESP8266, và thiết lập màn hình LCD. Nó cũng khởi tạo cảm biến DHT và in thông điệp khởi động lên màn hình serial. Hàm này đảm bảo rằng tất cả các cảm biến và giao diện giao tiếp đều sẵn sàng hoạt động.
- **Đọc data từ cảm biến** (arduino_main.ino)
  - Hàm lm35Method, với input là giá trị từ chân analog lm35Pin, chức năng là hiển thị nhiệt độ cơ thể trên LCD. Hàm này đọc nhiệt độ từ cảm biến LM35, tính toán nhiệt độ bằng độ C, và in giá trị lên màn hình serial. Nó

cũng hiển thị nhiệt độ trên màn hình LCD.

  - Hàm pulseSensorMethod, có input là giá trị chân PulseWire, với chức năng hiển thị nhịp tim (BPM) cơ thể trên LCD. Hàm này đọc nhịp tim từ cảm biến nhịp tim. Nếu phát hiện nhịp tim, in giá trị BPM lên màn hình serial, và hiển thị nó trên màn hình LCD.
  - Hàm dht11Sensor này đọc độ ẩm và nhiệt độ từ cảm biến DHT11. Nó in các giá trị lên màn hình serial và hiển thị chúng trên màn hình LCD. Nếu các giá trị đọc được hợp lệ, nó cập nhật màn hình LCD với các giá trị độ ẩm và nhiệt độ.

- **Giao tiếp với ESP8266** (arduino_main.ino)
  - Hàm esp8266Module gửi dữ liệu cảm biến đến module ESP8266 để xuất bản MQTT. Nó kiểm tra xem dữ liệu cảm biến có hợp lệ không, in dữ liệu lên màn hình serial, và gửi dữ liệu qua module ESP8266.
- **Main Loop** (arduino_main.ino)
  - Hàm loop là hàm chính chạy lặp lại liên tục. Nó đặt lại các giá trị cảm biến, kiểm tra kết nối MQTT, đọc dữ liệu từ đầu vào serial, xử lý dữ liệu nhận được, và gửi dữ liệu cảm biến đến MQTT broker. Nó cũng gọi các hàm để đọc dữ liệu từ cảm biến nhịp tim, cảm biến DHT11, và cảm biến LM35, và gửi dữ liệu qua module ESP8266. Hàm này bao gồm các khoảng trễ để đảm bảo thời gian giữa các lần đọc cảm biến và truyền dữ liệu được chính xác.

5.2.2 Giao tiếp với MQTT server

- **Setup** (esp8266_mqtt.ino)
  - setup: Hàm setup khởi tạo các thành phần khác nhau được sử dụng trong dự án. Nó thiết lập giao tiếp serial để phục vụ cho mục đích gỡ lỗi, khởi tạo kết nối WiFi bằng cách gọi hàm setup_wifi, và cấu hình client MQTT bằng cách đặt máy chủ và cổng. Ngoài ra, nó còn thiết lập client WiFi bảo mật cho giao tiếp MQTT. Hàm này đảm bảo rằng kết nối WiFi, client MQTT, và giao tiếp serial đều sẵn sàng để hoạt động.
  - setup_wifi: Hàm này thiết lập kết nối WiFi cho module ESP8266. Nó đặt chế độ WiFi thành chế độ station, bắt đầu kết nối với SSID và mật khẩu đã chỉ định, và chờ cho đến khi kết nối được thiết lập. Khi đã kết nối, nó sẽ in địa chỉ IP cục bộ lên màn hình serial.

- **MQTT** (esp8266_mqtt.ino)

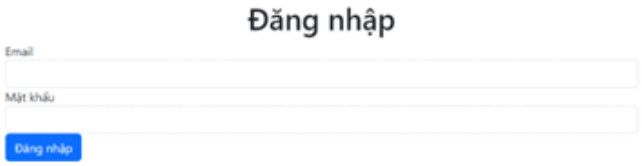
  - reconnect: Hàm này xử lý việc kết nối lại với MQTT broker nếu client bị ngắt kết nối. Nó cố gắng kết nối bằng cách sử dụng một ID client được tạo ngẫu nhiên và tên người dùng cùng mật khẩu MQTT đã cung cấp. Nếu kết nối thất bại, nó sẽ chờ 5 giây trước khi thử lại.
  - publishMessage: Hàm này gửi một thông điệp đến một chủ đề MQTT cụ thể. Nó nhận chủ đề và nội dung thông điệp làm tham số, in chủ đề và nội dung thông điệp lên màn hình serial, và gửi thông điệp đến MQTT broker.

- **Xử lý dữ liệu** (esp8266_mqtt.ino)
  - processReceivedData: Hàm này xử lý dữ liệu nhận được từ đầu vào serial. Nó kiểm tra tiền tố của dữ liệu để xác định loại dữ liệu cảm biến (BPM từ cảm biến nhịp tim, độ ẩm, nhiệt độ từ DHT11, hoặc nhiệt độ từ LM35) và cập nhật các biến tương ứng với các giá trị nhận được.
  - processMessage: Hàm này chuẩn bị dữ liệu cảm biến để gửi lên MQTT broker. Nó kiểm tra xem dữ liệu cảm biến có hợp lệ không, tạo một tài liệu JSON, và thêm dữ liệu cảm biến (nhiệt độ từ DHT11, độ ẩm từ DHT11, nhiệt độ từ LM35, và BPM) vào tài liệu JSON. Tài liệu JSON sau đó được tuần tự hóa thành một chuỗi để gửi đi.

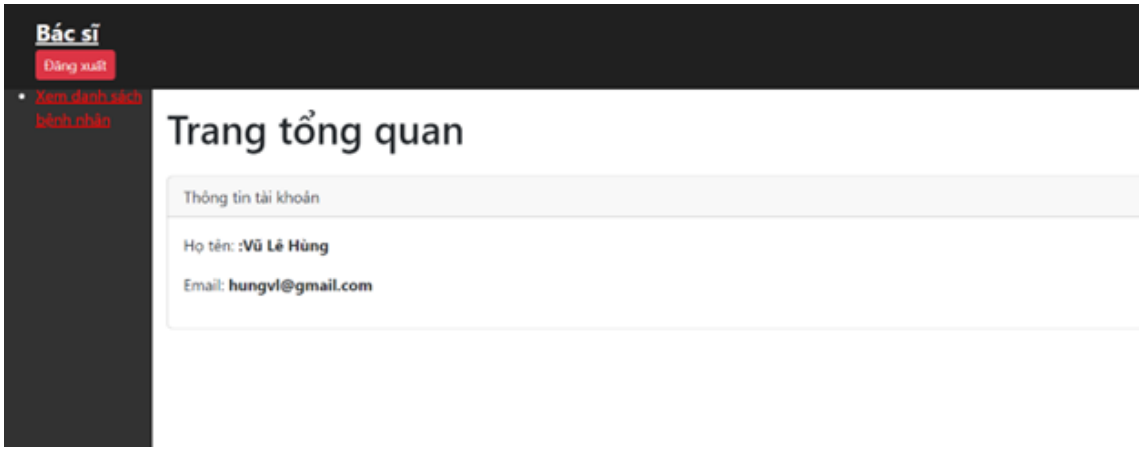
- **Main Loop** (esp8266_mqtt.ino)
  - loop: Đây là hàm chính chạy lặp lại liên tục. Nó đặt lại các giá trị cảm biến, kiểm tra kết nối MQTT, đọc dữ liệu từ đầu vào serial, xử lý dữ liệu nhận được, chuẩn bị dữ liệu cảm biến để gửi đi, và gửi dữ liệu cảm biến lên MQTT broker. Hàm này bao gồm một cờ để đảm bảo chỉ dữ liệu hợp lệ mới được gửi đi.

5.2.3 Thiết kế và cài đặt giao diện

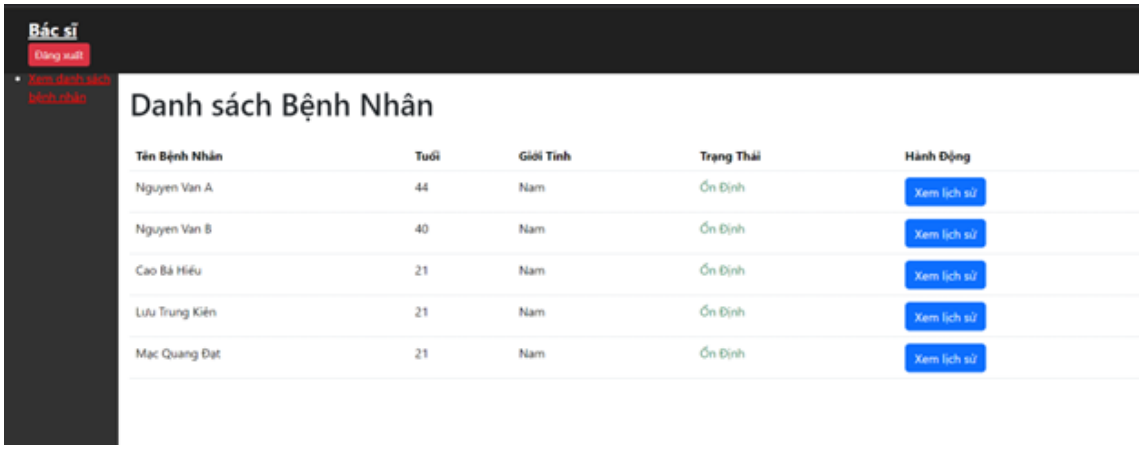
a, Giao diện các trang của nhân viên quản trị y tế



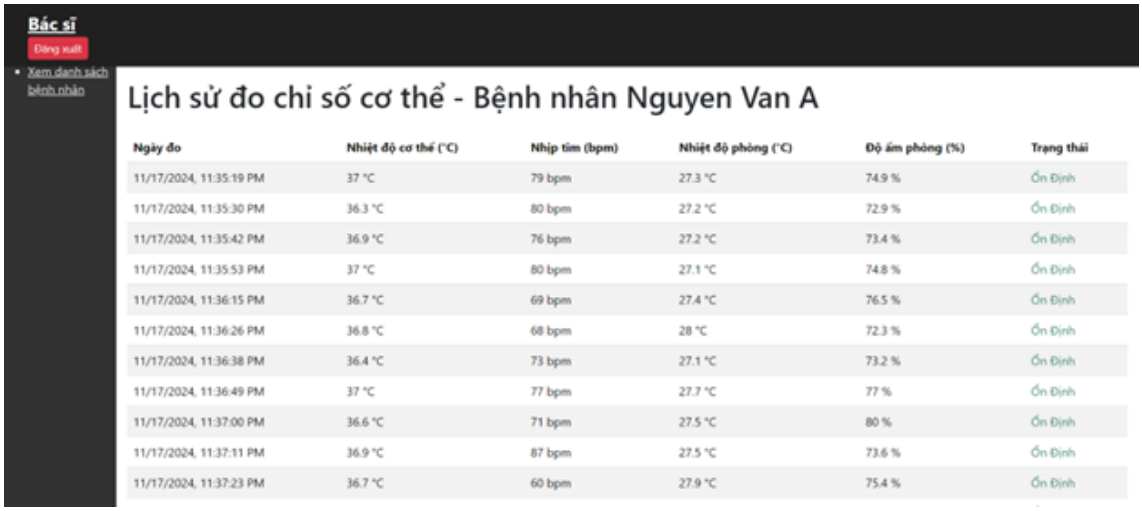
Hình 5.2: Giao diện đăng nhập cho nhân viên quản trị y tế



Hình 5.3: Giao diện trang chính của nhân viên quản trị y tế



Hình 5.4: Giao diện danh sách các bệnh nhân

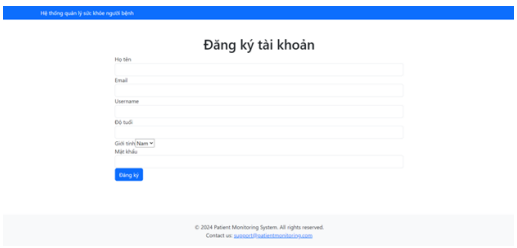


Hình 5.5: Giao diện lịch sử đo của một bệnh nhân cụ thể

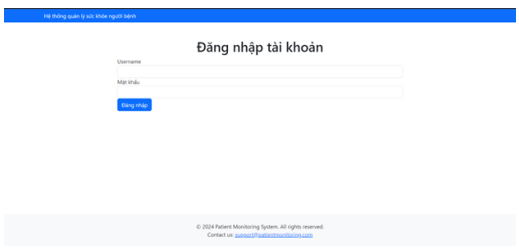
b, Giao diện các trang của bệnh nhân



Hình 5.6: Giao diện tổng quan ban đầu

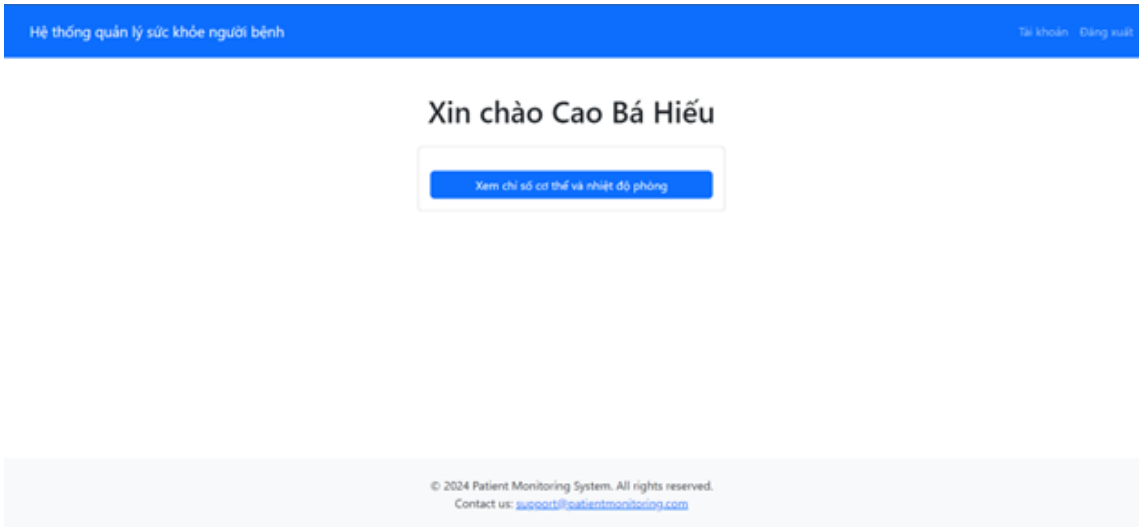


(a) Giao diện đăng kí cho bệnh nhân



(b) Giao diện đăng nhập cho bệnh nhân

Hình 5.7: Giao diện đăng nhập/đăng ký



Hình 5.8: Giao diện trang chính bệnh nhân



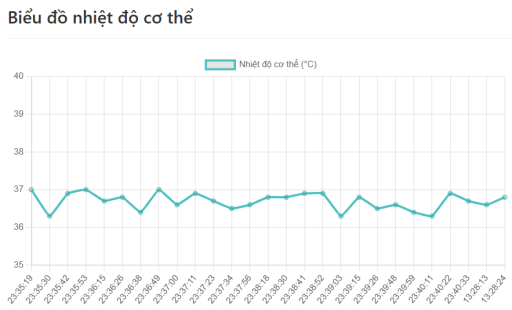
Hình 5.9: Giao diện trang thay đổi thông tin bệnh nhân



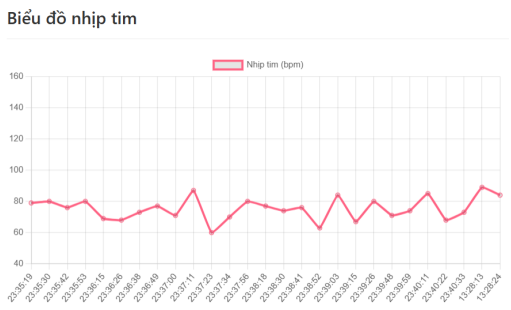
Hình 5.10: Giao diện trang chỉ số sức khỏe bệnh nhân



Hình 5.11: Giao diện trang hiện lịch sử đo chỉ số cơ thể bệnh nhân

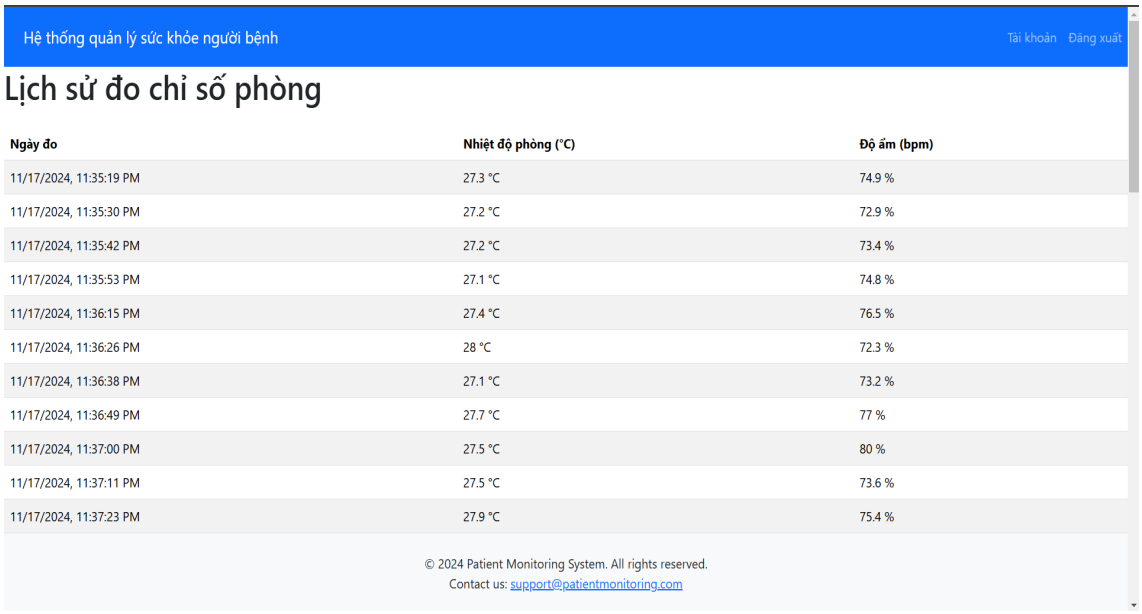


(a) Giao diện đồ thị hiển thị lịch sử đo nhiệt độ cơ thể bệnh nhân

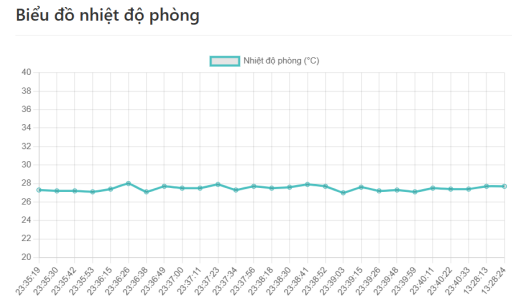


(b) Giao diện đồ thị hiển thị lịch sử đo nhịp tim bệnh nhân

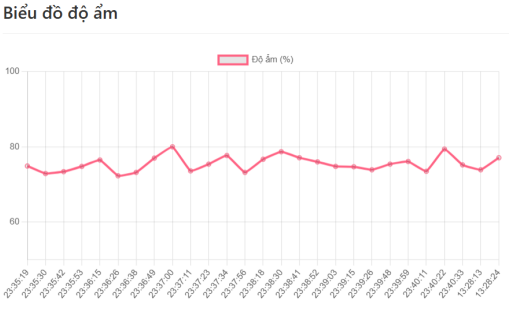
Hình 5.12: Giao diện đồ thị đo chỉ số bệnh nhân



Hình 5.13: Giao diện trang hiện lịch sử đo chỉ số phòng của bệnh nhân



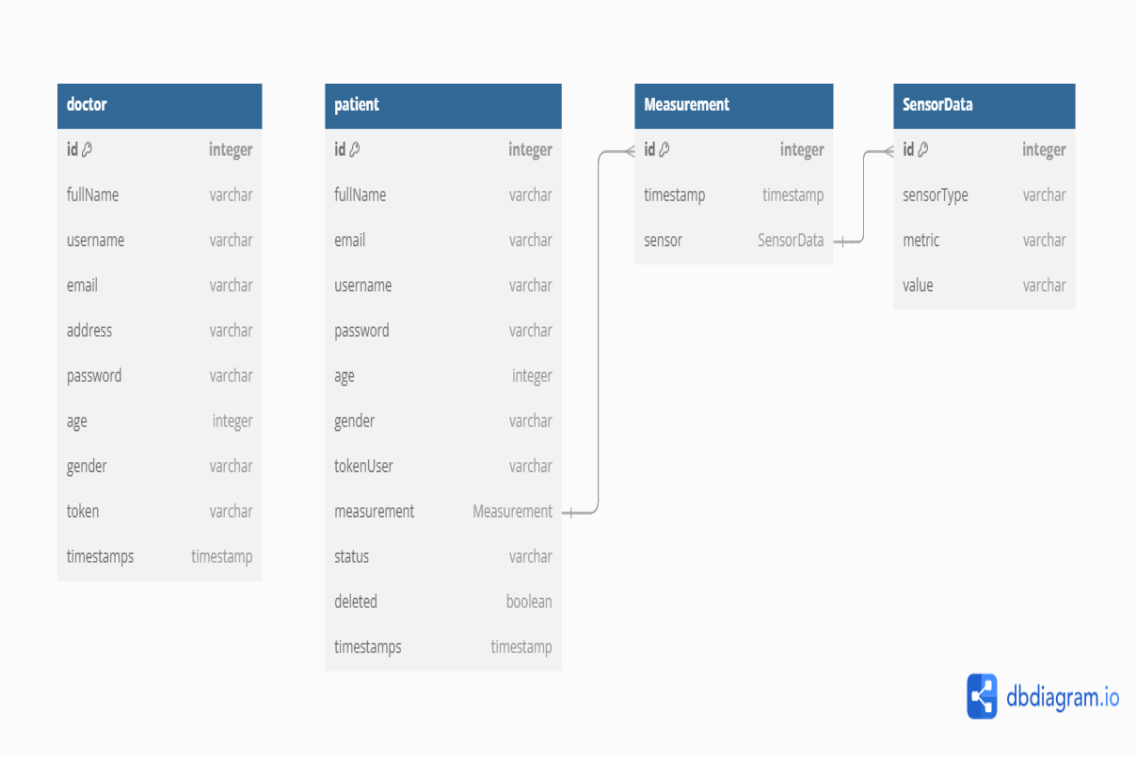
(a) Giao diện đồ thị hiển thị lịch sử đo nhiệt độ phòng



(b) Giao diện đồ thị hiển thị lịch sử đo độ ẩm phòng

Hình 5.14: Giao diện đồ thị đo chỉ số phòng bệnh

5.2.4 Cài đặt, kết nối và các phương thức xử lý liên quan tới database



Hình 5.15: Sơ đồ cơ sở dữ liệu

- **connect()** (config/database.js) Hàm này thiết lập kết nối với cơ sở dữ liệu MongoDB bằng cách sử dụng chuỗi kết nối được cung cấp trong các biến môi trường. Nó sử dụng phương thức mongoose.connect để kết nối với cơ sở dữ liệu và ghi lại thông báo thành công khi kết nối thành công hoặc thông báo lỗi nếu kết nối thất bại.
- **registerPost()** (controller/user_controller.js) Hàm này xử lý việc đăng ký bệnh nhân mới. Nó kiểm tra xem email hoặc tên người dùng đã tồn tại trong cơ sở dữ liệu hay chưa. Nếu chưa tồn tại, nó tạo một tài liệu Bệnh nhân mới với dữ liệu được cung cấp và lưu vào cơ sở dữ liệu. Sau đó, nó thiết lập một cookie với mã thông báo của bệnh nhân và chuyển hướng đến trang chào mừng.
- **loginPost()** (controller/user_controller.js) Hàm này xử lý quá trình đăng nhập cho bệnh nhân. Nó kiểm tra xem tên người dùng được cung cấp có tồn tại và mật khẩu có khớp với mật khẩu đã lưu hay không. Nếu thông tin đăng nhập chính xác, nó thiết lập ID người dùng toàn cầu, thiết lập một cookie với mã thông báo của bệnh nhân và chuyển hướng đến trang chào mừng. Nếu thông tin đăng nhập không chính xác, nó hiển thị thông báo lỗi và chuyển hướng trở lại trang đăng nhập.
- **edit()** (controller/user_controller.js) Hàm này cập nhật thông tin của bệnh

nhân. Nó kiểm tra xem email mới có tồn tại trong cơ sở dữ liệu cho bệnh nhân khác hay không. Nếu email không tồn tại, nó cập nhật thông tin của bệnh nhân với dữ liệu được cung cấp và hiển thị thông báo thành công. Sau đó, nó chuyển hướng trở lại trang trước đó.

- **roomMetricsHistory()** (controller/user_controller.js) Hàm này truy xuất lịch sử chỉ số phòng cho bệnh nhân. Nó duyệt qua các đo lường của bệnh nhân, truy xuất dữ liệu cảm biến tương ứng cho nhiệt độ phòng và độ ẩm, và biên soạn danh sách các chỉ số này. Sau đó, nó hiển thị trang lịch sử chỉ số phòng với dữ liệu đã biên soạn.
- **bodyMetricsHistory()** (controller/patient/user_controller.js) Hàm này truy xuất lịch sử chỉ số cơ thể cho bệnh nhân. Nó duyệt qua các đo lường của bệnh nhân, truy xuất dữ liệu cảm biến tương ứng cho nhiệt độ cơ thể và nhịp tim, và biên soạn danh sách các chỉ số này. Sau đó, nó hiển thị trang lịch sử chỉ số cơ thể với dữ liệu đã biên soạn.
- **deleteSensorData()** (controller/user_controller.js) Hàm này xóa tất cả dữ liệu cảm biến từ cơ sở dữ liệu. Nó sử dụng phương thức SensorData.deleteMany để xóa tất cả các tài liệu từ bộ sưu tập SensorData và gửi phản hồi thành công nếu thao tác thành công hoặc phản hồi lỗi nếu thất bại.
- **deleteMeasurement()** (controller/patient/user_controller.js) Hàm này xóa tất cả dữ liệu đo lường từ cơ sở dữ liệu. Nó sử dụng phương thức Measurement.deleteMany để xóa tất cả các tài liệu từ bộ sưu tập Measurement và gửi phản hồi thành công nếu thao tác thành công hoặc phản hồi lỗi nếu thất bại.
- **deleteMePatient()** (controller/patient/user_controller.js) Hàm này xóa tất cả các Measurement liên quan đến một bệnh nhân. Nó truy xuất tài liệu của bệnh nhân, xóa sạch mảng đo lường và lưu tài liệu đã cập nhật. Sau đó, nó gửi phản hồi thành công nếu thao tác thành công hoặc phản hồi lỗi nếu thất bại.
- **list()** (controller/doctor/patientListController.js) Hàm này truy xuất danh sách tất cả các bệnh nhân từ cơ sở dữ liệu. Nó sử dụng phương thức Patient.find để lấy tất cả các tài liệu bệnh nhân và hiển thị trang danh sách bệnh nhân với dữ liệu đã truy xuất.
- **history()** (controller/doctor/patientListController.js) Hàm này truy xuất lịch sử đo lường cho một bệnh nhân cụ thể. Nó duyệt qua các đo lường của bệnh nhân, truy xuất dữ liệu cảm biến tương ứng cho nhiệt độ phòng, độ ẩm, nhiệt độ cơ thể và nhịp tim, và biên soạn danh sách các chỉ số này. Sau đó, nó hiển thị trang lịch sử đo lường của bệnh nhân với dữ liệu đã biên soạn.

5.2.5 Cài đặt MQTT Client

- **initSocketIO()** (socket-io.js) Hàm này khởi tạo máy chủ Socket.IO với phiên bản máy chủ HTTP được cung cấp. Nó thiết lập một trình lắng nghe sự kiện kết nối để ghi lại khi người dùng kết nối hoặc ngắt kết nối. Hàm này đảm bảo rằng máy chủ Socket.IO sẵn sàng xử lý giao tiếp thời gian thực với các khách hàng đã kết nối.
- **getSocketIOInstance()** (socket-io.js) Hàm này trả về phiên bản Socket.IO đã được khởi tạo. Nếu phiên bản chưa được khởi tạo, nó sẽ in ra một lỗi. Hàm này cung cấp quyền truy cập vào phiên bản Socket.IO để phát ra các sự kiện và xử lý giao tiếp thời gian thực.
- **client.on("connect")** (mqtt_client.js) Trình xử lý sự kiện này được kích hoạt khi MQTT client kết nối thành công với HiveMQ Cloud. Nó lưu lại một thông báo thành công và đăng ký vào topic "sensor-data" để nhận các thông điệp dữ liệu cảm biến.
- **client.on("error")** (mqtt_client.js) Trình xử lý sự kiện này được kích hoạt khi có lỗi với MQTT client. Nó ghi lại thông báo lỗi vào bảng điều khiển. Hàm này giúp gỡ lỗi và xử lý các vấn đề kết nối với MQTT broker.
- **client.on("message")** (mqtt_client.js) Trình xử lý sự kiện này xử lý các thông điệp MQTT đến trên topic đã đăng ký. Nó phân tích thông điệp để trích xuất dữ liệu cảm biến, tạo các tài liệu SensorData mới và lưu chúng vào cơ sở dữ liệu. Nó cũng tạo một Measurement mới với các tham chiếu đến dữ liệu cảm biến đã lưu và thêm nó vào mảng Measurement của bệnh nhân. Hàm này phát ra dữ liệu cảm biến tới các khách hàng đã kết nối qua Socket.IO để cập nhật thời gian thực.

5.2.6 Các API cho các chức năng của nhân viên quản trị y tế

- **Login**
  - GET /doctor/auth/login
    - File code: auth_controller.js và auth_router.js
    - Input: None
    - Output: Hiển thị trang đăng nhập cho bác sĩ
    - Miêu tả: API này hiển thị trang đăng nhập cho bác sĩ. Nó kiểm tra xem có token trong cookie hay không. Nếu tìm thấy token, nó sẽ chuyển hướng bác sĩ đến trang bảng điều khiển. Nếu không tìm thấy token, nó sẽ hiển thị trang đăng nhập để bác sĩ nhập thông tin đăng nhập.

  - POST /doctor/auth/login
    - File code: auth_controller.js và auth_router.js
    - Input: email, password
    - Output: Chuyển hướng đến /doctor/dashboard khi đăng nhập thành công, hoặc chuyển về trang đăng nhập bác sĩ khi thất bại
    - Miêu tả: API này xử lý quá trình đăng nhập cho bác sĩ. Nó lấy email và mật khẩu từ thân yêu cầu, xác minh thông tin đăng nhập với cơ sở dữ liệu và đặt cookie chứa token của bác sĩ nếu thông tin đăng nhập chính xác. Nếu thông tin đăng nhập không chính xác, nó sẽ hiển thị thông báo lỗi và chuyển hướng lại trang đăng nhập.

- **Logout**
  - GET /doctor/auth/logout
    - File code: auth_controller.js và auth_router.js
    - Input: None
    - Output: Xóa phiên làm việc của bác sĩ và chuyển hướng về trang login bác sĩ
    - Miêu tả: API này xử lý quá trình đăng xuất cho bác sĩ. Nó xóa cookie chứa token và chuyển hướng bác sĩ đến trang đăng nhập, đảm bảo rằng bác sĩ đã đăng xuất và phải đăng nhập lại để truy cập các tuyến bảo vệ.

- **Dashboard**
  - GET /doctor/dashboard
    - File code: dashboard_controller.js và dashboard_router.js
    - Input: None
    - Output: Hiện trang dashboard quản lý của bác sĩ
    - Miêu tả: API này hiển thị trang bảng điều khiển cho bác sĩ. Nó yêu cầu xác thực và sử dụng token từ cookie để lấy thông tin của bác sĩ từ cơ sở dữ liệu. Trang bảng điều khiển hiển thị thông tin tài khoản và các dữ liệu liên quan khác của bác sĩ.

- **Patient List**
  - GET /doctor/patient-list
    - File code: patientList_controller.js và patientRouter.js

    - Input: None
    - Output: Hiện trang bao gồm danh sách các bệnh nhân
    - Miêu tả: API này lấy danh sách tất cả bệnh nhân từ cơ sở dữ liệu và hiển thị trang danh sách bệnh nhân. Nó yêu cầu xác thực và hiển thị tên, tuổi, giới tính và trạng thái của các bệnh nhân. Trang này cũng cung cấp các liên kết để xem lịch sử đo lường của từng bệnh nhân.
  - GET /doctor/patient-list/patient/:id/history
    - File code: patientList_controller.js và patientRouter.js
    - Input: id bệnh nhân
    - Output: Hiện trang bao gồm lịch sử data của bệnh nhân
    - Miêu tả: API này lấy lịch sử đo lường của một bệnh nhân cụ thể. Nó yêu cầu xác thực và sử dụng ID bệnh nhân từ tham số URL để lấy dữ liệu đo lường của bệnh nhân từ cơ sở dữ liệu. Điểm cuối này tổng hợp dữ liệu thành danh sách các chỉ số, bao gồm nhiệt độ cơ thể, nhịp tim, nhiệt độ phòng và độ ẩm, và hiển thị trang lịch sử đo lường của bệnh nhân với dữ liệu đã tổng hợp.

5.2.7 Các API cho các chức năng của bệnh nhân

- **Register**
  - GET /Register
    - File code: user_controller.js, user_validate.js và user_router.js
    - Input: None
    - Output: Hiển thị trang đăng kí
  - POST /Register
    - File code: user_controller.js, user_validate.js và user_router.js
    - Input: fullName, email, password, username, age
    - Output: Chuyển hướng đến /user/welcome khi đăng kí thành công, hoặc chuyển về trang đăng kí khi thất bại
    - Miêu tả: API này hiển thị trang đăng ký cho bệnh nhân. Nó cho phép bệnh nhân mới nhập thông tin cá nhân như họ tên, email, tên đăng nhập, mật khẩu, tuổi và giới tính để tạo tài khoản mới.
- **Login**

  - GET /Login
    - File code: user_controller.js, user_validate.js và user_router.js
    - Input: None
    - Output: Hiển thị trang đăng nhập
    - Miêu tả: API này hiển thị trang đăng nhập cho bệnh nhân. Nó cho phép bệnh nhân hiện tại nhập tên đăng nhập và mật khẩu để đăng nhập vào tài khoản của họ.
  - POST /Login
    - File code: user_controller.js, user_validate.js và user_router.js
    - Input: username, password
    - Output: API này xử lý quá trình đăng nhập cho bệnh nhân. Nó xác thực dữ liệu đầu vào, kiểm tra thông tin đăng nhập với cơ sở dữ liệu, và thiết lập cookie với mã thông báo của bệnh nhân nếu thông tin đăng nhập chính xác. Nếu thông tin đăng nhập không chính xác, nó sẽ hiển thị thông báo lỗi và chuyển hướng lại trang đăng nhập.

- **Logout**
  - GET /Logout
    - File code: user_controller.js và user_router.js
    - Input: None
    - Output: Xóa phiên người dùng và chuyển về trang chính trước khi đăng nhập
    - Miêu tả: API này xử lý quá trình đăng xuất cho bệnh nhân. Nó xóa cookie mã thông báo và chuyển hướng bệnh nhân về trang chủ, đảm bảo rằng bệnh nhân đã đăng xuất và phải đăng nhập lại để truy cập các trang bảo vệ.

- **Welcome**
  - GET /welcome
    - File code: user_controller.js, auth_middleware.js và user_router.js
    - Input: None
    - Output: Hiện trang chủ sau khi đăng nhập hoặc đăng kí thành công
    - Miêu tả: API này hiển thị trang chào mừng cho bệnh nhân đã đăng

nhập. Nó hiển thị thông điệp chào mừng và cung cấp các liên kết để xem chỉ số cơ thể, chỉ số phòng và thông tin tài khoản.

- **Account information**
  - GET /tai-khoan
    - File code: user_controller.js và user_router.js
    - Input: None
    - Output: Hiển thị thông tin tài khoản
    - Miêu tả: API này hiển thị trang thông tin tài khoản cho bệnh nhân. Nó hiển thị thông tin cá nhân của bệnh nhân và cho phép họ cập nhật chi tiết.
  - POST /tai-khoan/edit
    - File code: user_controller.js và user_router.js
    - Input: fullName, email, age, gender
    - Output: Quay lại trang thông tin tài khoản khi thành công hoặc hiển thị lỗi khi không thành công
    - Miêu tả: API này xử lý quá trình cập nhật thông tin tài khoản của bệnh nhân. Nó xác thực dữ liệu đầu vào, kiểm tra email đã tồn tại trong cơ sở dữ liệu hay chưa, và cập nhật hồ sơ bệnh nhân nếu dữ liệu hợp lệ. Sau đó, nó hiển thị thông báo thành công và chuyển hướng lại trang thông tin tài khoản.
- **Metrics**
  - GET /metrics
    - File code: user_controller.js và user_router.js
    - Input: None
    - Output: Hiển thị các thông số sức khỏe
    - Miêu tả: API này hiển thị trang chỉ số cho bệnh nhân. Nó hiển thị chỉ số cơ thể và chỉ số phòng của bệnh nhân theo thời gian thực, sử dụng dữ liệu nhận từ các cảm biến.
- **Body metrics history**
  - GET /body-metrics/history
    - File code: user_controller.js và user_router.js

    - Input: None
    - Output: Hiển thị trang lịch sử với các thông số sức khỏe người bệnh
    - Miêu tả: API này lấy lịch sử chỉ số cơ thể cho bệnh nhân. Nó tổng hợp danh sách các đo lường nhiệt độ cơ thể và nhịp tim từ cơ sở dữ liệu và hiển thị trang lịch sử chỉ số cơ thể với dữ liệu đã tổng hợp.

- **Room metrics history**
  - GET /room-metrics/history
    - File code: user_controller.js và user_router.js
    - Input: None
    - Output: Hiển thị trang lịch sử với các thông số nhiệt độ và độ ẩm phòng
    - Miêu tả: API này lấy lịch sử chỉ số phòng cho bệnh nhân. Nó tổng hợp danh sách các đo lường nhiệt độ phòng và độ ẩm từ cơ sở dữ liệu và hiển thị trang lịch sử chỉ số phòng với dữ liệu đã tổng hợp.

- **Delete sensor data**
  - GET /delete-sensor
    - File code: user_controller.js và user_router.js
    - Input: None
    - Output: Xóa thông tin thu thập được từ cảm biến và trả về message
    - Miêu tả: API này xóa tất cả dữ liệu cảm biến từ cơ sở dữ liệu. Nó sử dụng phương thức SensorData.deleteMany để xóa tất cả tài liệu từ bộ sưu tập SensorData và gửi phản hồi thành công nếu thao tác thành công hoặc phản hồi lỗi nếu thất bại.

- **Delete measurement**
  - GET /delete-measure
    - File code: user_controller.js và user_router.js
    - Input: None
    - Output: Xóa chuỗi data thu thập được từ cảm biến và trả về message
    - Miêu tả: API này xóa tất cả dữ liệu đo lường từ cơ sở dữ liệu. Nó sử dụng phương thức Measurement.deleteMany để xóa tất cả tài liệu từ bộ sưu tập Measurement và gửi phản hồi thành công nếu thao tác

thành công hoặc phản hồi lỗi nếu thất bại.

- **Delete patient measurements**
  - GET /delete-measure-patient
    - File code: user_controller.js và user_router.js
    - Input: None
    - Output: Xóa dữ liệu đo được của một bệnh nhân và trả về message
    - Miêu tả: API này xóa tất cả các đo lường liên quan đến một bệnh nhân. Nó lấy tài liệu của bệnh nhân, xóa mảng đo lường và lưu tài liệu đã cập nhật. Sau đó, nó gửi phản hồi thành công nếu thao tác thành công hoặc phản hồi lỗi nếu thất bại.

5.3 Bảng phân chia công việc

5.3.1 Phân công ban đầu

Họ và tên	Mã Sinh Viên	Các nội dung tham gia
Mạc Quang Đạt	B21DCCN215	Viết báo cáo Lập trình Arduino nhận dữ liệu từ cảm biến Rà soát và fix bug
Vũ Lê Hùng	B21DCCN059	Viết báo cáo Mua và chuẩn bị các thiết bị Lắp đặt các thiết bị
Cao Bá Hiếu	B21DCCN347	Lập trình Back End Tìm hiểu và lập trình giao thức MQTT Lập trình Arduino nhận dữ liệu từ cảm biến Lắp đặt các thiết bị
Lưu Trung Kiên	B21DCCN071	Lập trình Front End Cài đặt và xử lý database Rà soát và fix bug

Bảng 5.1: Bảng thông tin thành viên và công việc.

5.3.2 Theo dõi tiến trình

Công việc	Bắt đầu	Hoàn thành	Người thực hiện
Chuẩn bị kiến thức và thiết bị			
Tìm hiểu về Arduino và cách lập trình Arduino	10/09	12/09	Cả nhóm
Mua các thiết bị	15/09	18/09	Vũ Lê Hùng
Tìm hiểu về giao thức MQTT	15/09	20/09	Cao Bá Hiếu

Lắp đặt các thiết bị	20/09	06/10	Vũ Lê Hùng Cao Bá Hiếu
Hoàn thiện sản phẩm			
Lập trình Arduino nhận dữ liệu từ cảm biến	06/10	12/10	Cao Bá Hiếu Mạc Quang Đạt
Lập trình gửi và nhận dữ liệu với giao thức MQTT	12/10	14/10	Cao Bá Hiếu
Lập trình Frontend website	20/10	27/10	Lưu Trung Kiên
Lập trình Backend website	20/10	27/10	Cao Bá Hiếu
Cài đặt và xử lý database	28/10	30/10	Lưu Trung Kiên
Rà soát và fix bug	1/11	3/11	Lưu Trung Kiên Mạc Quang Đạt
Viết báo cáo	10/11	20/11	Mạc Quang Đạt Vũ Lê Hùng

Bảng 5.2: Bảng theo dõi tiến trình.

CHƯƠNG 6. ĐÁNH GIÁ KẾT QUẢ DỰ ÁN, KẾT LUẬN

6.1 Đánh giá kết quả dự án

6.1.1 Các yêu cầu chi tiết theo nhóm người dùng đã hoàn thành

a, Quản trị viên nhân viên y tế

- Front-end
  1. Bảng điều khiển tổng quan, hiển thị thông tin hệ thống, số lượng người dùng.
  2. Xem thông tin bệnh nhân, lịch sử các chỉ số sức khỏe và chỉ số phòng bệnh.
- Back-end
  1. API quản lý bệnh nhân, dữ liệu từ cảm biến.
  2. Cơ sở dữ liệu lưu trữ thông tin người dùng, thiết bị, dữ liệu từ các cảm biến.

b, Bệnh nhân

- Front-end
  1. Xem được thông tin cá nhân
  2. Xem được thông tin sức khỏe cá nhân, lịch sử đo, các chỉ số sinh tồn được đo theo thời gian thực.
  3. Xem được các chỉ số sinh tồn dưới dạng bảng và đồ thị.
- Back-end
  1. Thay đổi được thông tin cá nhân.
  2. API để lấy dữ liệu mới đo được từ các cảm biến.
  3. API để lấy dữ liệu cá nhân, các dữ liệu đã đo trước đó.

c, Thiết bị

- Cảm biến nhịp tim
  - Đo nhịp tim trong khoảng 20-200 nhịp/phút.
  - Nếu nhịp tim ngoài khoảng này, in lỗi thiết bị và không gửi lên server.
- Cảm biến nhiệt độ
  - Đo nhiệt độ trong khoảng từ 20°C đến 50°C.

- Nếu nhiệt độ ngoài khoảng này, in lỗi thiết bị và không gửi lên server.

- Cảm biến DHT11
  - Đo độ ẩm trong khoảng từ 20% đến 90%.
  - Nếu độ ẩm ngoài khoảng này, in lỗi thiết bị và không gửi lên server.
  - Đo nhiệt độ trong khoảng từ 20°C đến 50°C.
  - Nếu nhiệt độ ngoài khoảng này, in lỗi thiết bị và không gửi lên server.

6.1.2 Các yêu cầu chi tiết theo nhóm người dùng chưa hoàn thành

a, Quản trị viên nhân viên y tế

- Front-end
  1. Chưa hiển thị được các thiết bị kết nối.
  2. Chưa có cảnh báo sức khỏe của bệnh nhân.

- Back-end
  1. Chưa cài đặt được API cảnh báo, chưa thể thiết lập ngưỡng cảnh báo, xem lịch sử cảnh báo.
  2. Chưa thể phân quyền người dùng.
  3. Chưa xem được danh sách thiết bị kết nối.

b, Bệnh nhân

- Front-end
  1. Chưa hiển thị được cảnh báo sức khỏe.

c, Thiết bị

- Cảm biến nhịp tim
  - Chưa cài được ngưỡng cảnh báo nếu nhịp tim nhỏ hơn 60 nhịp/phút hoặc lớn hơn 120 nhịp/phút.

- Cảm biến nhiệt độ
  - Chưa cài được ngưỡng cảnh báo nếu nhịp tim nhỏ hơn 35 độ C hoặc lớn hơn 39 độ C.

- Cảm biến DHT11
  - Chưa cài được ngưỡng cảnh báo nếu nhiệt độ nhỏ hơn 15 độ C hoặc lớn hơn 35 độ C.

6.1.3 Kết luận

Trong quá trình triển khai, hệ thống IoT giám sát sức khỏe bệnh nhân đã chứng minh khả năng thu thập và xử lý hiệu quả các dữ liệu sinh trắc quan trọng như nhịp tim, nhiệt độ cơ thể, độ ẩm và nhiệt độ phòng. Hệ thống đã thành công trong việc đo lường các chỉ số sinh tồn, hỗ trợ nhân viên y tế giám sát sức khỏe bệnh nhân một cách chính xác và kịp thời. Tuy nhiên, trong quá trình thử nghiệm, dự án phát sinh một số hạn chế về độ chính xác của cảm biến nhiệt độ cơ thể, cảm biến nhịp tim và khả năng tích hợp với các thiết bị y tế khác. Những hạn chế này sẽ được tập trung cải thiện trong các phiên bản tiếp theo nhằm nâng cao chất lượng dịch vụ và hỗ trợ tốt hơn cho quá trình điều trị bệnh nhân.

Dự án này không chỉ hoàn thiện về mặt kỹ thuật mà còn mang lại những giá trị thiết thực trong việc ứng dụng công nghệ IoT vào chăm sóc sức khỏe. Thông qua hệ thống giám sát sức khỏe bệnh nhân, bài toán theo dõi và quản lý tình trạng sức khỏe có thể được giải quyết một cách toàn diện và tối ưu. Những bài học và kinh nghiệm có được từ dự án sẽ là nền tảng vững chắc cho các dự án công nghệ IoT khác trong tương lai.

6.1.4 Rút ra bài học kinh nghiệm

Quá trình phát triển dự án IoT giám sát sức khỏe bệnh nhân đã mang lại cho nhóm nhiều kinh nghiệm quý báu thông qua những thử thách và trải nghiệm thực tế. Dự án này đã mang lại những bài học quan trọng:

- **Lập kế hoạch chi tiết:** Một trong những bài học quan trọng nhất là sự cần thiết của việc lập kế hoạch chi tiết ngay từ đầu và phân bổ thời gian hợp lý cho từng giai đoạn phát triển. Thiếu kế hoạch chi tiết có thể dẫn đến chậm tiến độ, thiếu sót trong việc hoàn thành các yêu cầu kỹ thuật hoặc không đảm bảo chất lượng sản phẩm cuối cùng.
- **Kiểm thử và tối ưu hóa liên tục:** Dự án đã chứng minh rằng việc kiểm thử liên tục và tối ưu hóa phần mềm và thiết bị là rất quan trọng, đặc biệt khi triển khai các hệ thống IoT với nhiều thiết bị kết nối. Các buổi kiểm thử thường xuyên không chỉ giúp phát hiện sớm các lỗi tiềm ẩn mà còn giúp tối ưu hóa mã nguồn và cấu hình hệ thống, đảm bảo hệ thống vận hành ổn định và hiệu quả.
- **Giải quyết vấn đề và tích hợp hệ thống:** Trong quá trình phát triển, nhóm đã phải đối mặt với nhiều khó khăn trong việc cấu hình thiết bị, tích hợp dữ liệu từ các cảm biến và đồng bộ hóa giữa các hệ thống khác nhau. Những thử thách này đã giúp nhóm rèn luyện khả năng giải quyết vấn đề sáng tạo, linh

hoạt và nâng cao kỹ năng làm việc dưới áp lực. Đây là kỹ năng vô giá không chỉ trong việc triển khai các hệ thống IoT mà còn trong việc giải quyết các vấn đề kỹ thuật phức tạp.

- **Giao tiếp và phối hợp nhóm:** Quá trình triển khai dự án đòi hỏi nhóm phải có khả năng giao tiếp hiệu quả và phối hợp nhịp nhàng giữa các thành viên. Việc chia sẻ thông tin, thảo luận ý tưởng và giải quyết vấn đề cùng nhau không chỉ giúp tăng cường sự gắn kết trong nhóm mà còn nâng cao năng suất làm việc. Kỹ năng làm việc nhóm và sự phối hợp tốt giữa các thành viên là yếu tố quan trọng trong việc đảm bảo sự thành công của dự án.

6.1.5 Định hướng phát triển tương lai

Để hệ thống IoT giám sát sức khỏe bệnh nhân phát huy tối đa tiềm năng, nhóm đã xác định một số hướng phát triển trọng tâm sau:

- **Tối ưu hóa việc thu thập và phân tích dữ liệu sinh trắc**
  - Cảm biến đa dạng: Tích hợp thêm các loại cảm biến để thu thập đa dạng các chỉ số sinh trắc như nhịp tim, huyết áp, nhịp thở, nhiệt độ cơ thể... giúp theo dõi sức khỏe toàn diện hơn.
  - Xử lý dữ liệu thời gian thực: Dữ liệu thời gian thực là nguồn dữ liệu quan trọng trong việc phát hiện sớm các dấu hiệu bất thường và đưa ra cảnh báo kịp thời.
  - Học máy: Sử dụng các mô hình học máy để xây dựng các thuật toán dự đoán, giúp phát hiện các xu hướng bệnh lý tiềm ẩn và đưa ra các khuyến nghị về lối sống, chế độ ăn uống phù hợp.
- **Giao diện người dùng thân thiện và trực quan**
  - Ứng dụng di động: Phát triển ứng dụng di động để bệnh nhân và người nhà có thể theo dõi tình trạng sức khỏe mọi lúc mọi nơi, nhận thông báo và tương tác với hệ thống.
  - Bảng điều khiển: Thiết kế bảng điều khiển trực quan để các nhân viên y tế dễ dàng theo dõi và quản lý dữ liệu của nhiều bệnh nhân, đồng thời hỗ trợ việc ra quyết định điều trị.
- **Bảo mật và riêng tư dữ liệu**
  - Mã hóa dữ liệu: Áp dụng các phương pháp mã hóa mạnh mẽ để bảo vệ thông tin cá nhân của bệnh nhân.
  - Quản lý quyền truy cập: Thiết lập hệ thống quản lý quyền truy cập chặt chẽ, chỉ cho phép những người có thẩm quyền truy cập vào dữ liệu.

- **Tích hợp với hệ thống y tế**
  - Kết nối với hồ sơ sức khỏe điện tử: Tích hợp hệ thống với hồ sơ sức khỏe điện tử của bệnh nhân để có cái nhìn toàn diện về lịch sử bệnh án.
  - Liên kết với các thiết bị y tế khác: Kết nối với các thiết bị y tế khác như máy đo đường huyết, máy đo oxy máu để thu thập dữ liệu một cách đồng bộ.
- **Nâng cao khả năng mở rộng**
  - Kiến trúc hệ thống linh hoạt: Thiết kế hệ thống với kiến trúc mở, dễ dàng tích hợp thêm các tính năng và thiết bị mới trong tương lai.
  - Cơ sở dữ liệu hiệu quả: Sử dụng cơ sở dữ liệu có khả năng mở rộng để đáp ứng nhu cầu lưu trữ dữ liệu lớn.

Các hướng phát triển này sẽ giúp hệ thống IoT giám sát sức khỏe bệnh nhân trở nên thông minh hơn, hiệu quả hơn và đáp ứng tốt hơn nhu cầu của người dùng.

TÀI LIỆU THAM KHẢO

[1] *Getting started with arduino ide 2*, https://docs.arduino.cc/software/ide-v2/tutorials/getting-started-ide-v2/, Accessed: 10-10-2024.

[2] *Uno r3*, https://docs.arduino.cc/hardware/uno-rev3/, Accessed: 11-10-2024.

[3] *Esp8266 nodemcu*, https://nodemcu.readthedocs.io/en/release/getting-started/, Accessed: 12-10-2024.

[4] *Lm35*, https://www.ti.com/product/LM35, Accessed: 12-10-2024.

[5] *Dht11 humidity & temperature sensor*, 2021.

[6] *Pulse sensor getting started guide*, 2019.

[7] *Code bài tập lớn, github link*, https://github.com/Kaitokien/IoT-Web, Accessed: 22-11-2024.